Journal of *Cybersecurity and Privacy*

MDPI

Systematic Review

LLM-Based Agents for Cybersecurity: A Systematic Review of Architectures, Applications, and Open Challenges

George Fatouros [1,*], Konstantinos Mavrogiorgos [2,*], Georgios Makridis [2], John Soldatos [1] and Dimosthenis Kyriazis [2]

[1] Innov-Acts Ltd., 6 Kolokotroni Street, 1101 Nicosia, Cyprus; jsoldat@innov-acts.com
[2] Department of Digital Systems, University of Piraeus, 18534 Piraeus, Greece; gmakridis@unipi.gr (G.M.); dimos@unipi.gr (D.K.)
* Correspondence: gfatouros@innov-acts.com (G.F.); komav@unipi.gr (K.M.)

Abstract

The rapid evolution of Large Language Models (LLMs) has opened new frontiers in cybersecurity automation, enabling intelligent agents capable of multi-step reasoning, tool invocation, and autonomous decision-making across complex security tasks. While individual applications have emerged across threat intelligence, vulnerability assessment, penetration testing, and security operations center (SOC) automation, a systematic understanding of the LLM-based agent paradigm in cybersecurity—encompassing both single-agent and multi-agent architectures—remains lacking. This paper presents a systematic literature review following PRISMA guidelines, identifying records through 59 structured web-search queries whose results resolve predominantly to arXiv, Semantic Scholar, the ACM Digital Library, IEEE Xplore, USENIX, MDPI, SpringerLink, and Elsevier ScienceDirect, supplemented by citation chaining, for works published between January 2022 and April 2026; the full query record is published with the paper. We applied structured inclusion and exclusion criteria and classified 59 primary studies along five dimensions: security function, agent architecture pattern, knowledge augmentation strategy, human-in-the-loop posture, and evaluation rigor. Our analysis reveals that penetration testing and threat intelligence are the most extensively studied domains, while incident response and compliance verification remain critically underrepresented. Penetration testing alone accounts for over half the corpus (50.8%). Single-agent tool-calling remains the most prevalent architecture (30.5% of studies), whereas centralized multi-agent orchestration—present in 18.6%—yields the strongest reported performance gains, up to 4.3× on zero-day exploitation; prevalence and performance therefore point in opposite directions. No included study achieves production-grade (E4) evaluation: the entire field currently rests on controlled laboratory assessments. An independent search of six bibliographic databases recovers 86.3% of the studies the primary search had surfaced (79.7% of the full corpus) while indicating a total eligible literature of roughly 400 studies, so the corpus is reported as a documented subset rather than an exhaustive census. We propose a unifying taxonomy, identify cross-cutting challenges including hallucination, prompt injection, and benchmark fragmentation, and outline open research directions with particular emphasis on multi-agent orchestration design. Financial sector applicability under DORA and the EU AI Act is treated as a documented evidence gap rather than a synthesis: the corpus's only compliance and risk assessment study is also its only banking-specific system.

Check for updates

Academic Editor: Martin Gilje Jaatun

Received: 6 July 2026
Revised: 29 August 2026
Accepted: 6 September 2026
Published: 9 September 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

Keywords: large language models; cybersecurity agents; multi-agent systems; LLM orchestration; threat intelligence; vulnerability assessment; penetration testing; retrieval-augmented generation; security operations; systematic review

1. Introduction

The increasing volume, velocity, and sophistication of cyber threats have placed significant operational pressure on modern Security Operations Centers (SOCs), vulnerability management teams, and incident response units [1]. Traditional rule-based and signature-driven defenses are proving insufficient against adaptive adversaries, leading to alert fatigue, delayed triage, and escalating remediation costs [2]. Concurrently, Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding, code reasoning, and multi-step planning [3,4], creating a compelling opportunity to redesign cybersecurity workflows around intelligent, autonomous agents.

An LLM-based agent differs from a conventional LLM deployment in that it operates within an action loop: it perceives context, reasons over available tools, issues actions, observes outcomes, and iterates toward a goal [5,6]. In cybersecurity, this paradigm enables behaviors such as autonomous exploit chain construction, cyber threat intelligence (CTI) knowledge graph extraction, real-time alert triage, and adaptive incident response—tasks that previously required sustained expert engagement. Multi-agent extensions introduce further capability: specialized subagents operating under a central orchestrator can decompose complex security events into parallel, domain-specific analyses that a single generalist agent would otherwise have to perform sequentially [7].

Despite a growing body of individual system proposals, the field lacks a consolidated, methodology-driven synthesis of how LLM agents are being applied across the cybersecurity domain. Several recent surveys address adjacent ground. He et al. [8] treat the security and privacy of LLM agents as attack targets rather than as tools for cybersecurity work; Kaniewski et al. [9] focuses on a single application area, and Deng et al. [10]'s PentestGPT demonstrates what a single well-engineered system achieves in one function (penetration testing) rather than surveying the field. Closest to the present work, Srinivas et al. [1] apply a PRISMA-guided screening protocol to LLMs and agents for security automation with the security operations center as the organizing frame, and Shahriar et al. [11] document a comparably systematic search and screening process over a much larger, general agentic-security corpus; both stop short of publishing a per-paper coding matrix a reader can audit. Refs. [12–14] provide narrative overviews of agentic security published in 2025–2026.

What distinguishes this review is summarized in Table 1. None of the adjacent surveys publishes a reader-auditable, per-paper coding matrix: Srinivas et al. [1] states its screening criteria and funnel but tabulates only representative systems rather than its full 105-paper corpus, and Shahriar et al. [11] publishes a continuously updated list of its 260-plus papers without per-paper dimensional coding. We instead apply a documented retrieval and screening protocol, publish the full per-paper coding matrix across five dimensions, and span eight security functions rather than a single operational context or sub-domain. We additionally analyze the regulatory surface—the Digital Operational Resilience Act (DORA), the Network and Information Security Directive 2 (NIS2), and the EU AI Act—and audit retrieval-augmented systems for reporting completeness (Section 6), neither of which any adjacent survey treats. The contribution is therefore less the observation that LLM agents are being applied to security, which is by now uncontroversial, than a structured and auditable account of *where*, *how*, and *how well*.

Table 1. Positioning of this review against the closest adjacent surveys. SLR: Systematic literature review. "Corpus published" indicates whether a per-paper classification of the reviewed works is provided in a form a reader can audit or extend.

	[1]	[8]	[11]	[12]	**This Review**
Scope	SOC automation	Security and privacy of LLM agents as attack targets	Agentic security, general (applications, threats, defenses)	Agentic AI in cybersecurity (SOC, IR, offense, governance)	Eight security functions across the defensive lifecycle
Method	PRISMA-guided; stated inclusion criteria (105-paper funnel)	Narrative	Documented multi-stage search (DB query + venue scan + snowballing); stated inclusion/exclusion criteria	Narrative	Documented protocol with stated inclusion and exclusion criteria
Corpus published	No—representative systems cited, full 105-paper list not tabulated	No	Public, continuously updated list (more than 260 papers); not per-paper coded	No	Yes—59 studies, full coding matrix (Appendix B)
Architecture analysis	Partial—coarse per-system topology tags	No	Quantified—six paradigms with per-category counts, not broken out by security function	Conceptual	Six patterns, single- and multi-agent, with reported performance
HITL and evaluation rigor	Autonomy-maturity model (5 levels); not coded per security function	No	Quantified by offense/defense; not coded per security function	No	Coded dimensions (D4, D5), per function (Section 6)
Knowledge augmentation transparency	No	No	No	No	Audited—vector-store/embedding-model disclosure checked per study
Regulatory analysis	No	No	No	No	DORA, NIS2, EU AI Act

This paper makes the following contributions. The taxonomy is the primary contribution; the coded corpus is the evidence base that populates and tests it.

- A **five-dimensional taxonomy** for classifying LLM-based cybersecurity agents by security function, agent architecture, knowledge augmentation, human-in-the-loop posture, and evaluation rigor.
- A **PRISMA-guided review** of 59 primary studies published between 2022 and 2026, spanning eight security function categories, with the full search record and per-paper coding published for inspection. We characterize this corpus explicitly as a documented subset rather than a census: an independent search of six bibliographic databases recovers 86.3% of the studies the primary search had surfaced (79.7% of the full corpus) while indicating a total eligible literature on the order of 400 studies (Section 3.1). It is the evidence base for the taxonomy, not a claim of exhaustive coverage.
- A **comparative analysis** of agent architecture patterns, separating prevalence from performance: single-agent tool-calling dominates by count, while centralized multi-agent orchestration produces the strongest reported results from the smallest evidence base.
- A structured **gap analysis** revealing critical underrepresentation in incident response, forensics, and compliance verification.
- Identification of **open research directions**, including regulatory alignment under DORA [15], NIS2 [16], and the EU AI Act [17], where we document the near-absence of financial sector evidence rather than synthesizing it.

The remainder of this paper is organized as follows. Section 2 introduces the necessary background on LLM agent architectures and cybersecurity taxonomy. Section 3 describes the systematic review methodology and PRISMA flow. Section 4 covers LLM agent applications by security domain. Section 5 analyzes multi-agent orchestration architectures

in depth. Section 6 discusses cross-cutting challenges. Section 7 presents open research directions. Section 8 concludes the paper.

2. Background and Preliminaries

2.1. LLM Capabilities Relevant to Cybersecurity

Large Language Models are deep neural networks based on the transformer architecture [18], pre-trained on large corpora through next-token prediction and subsequently aligned to follow instructions via reinforcement learning from human feedback (RLHF) [19] or direct preference optimization [20]. Several capabilities of contemporary LLMs are particularly consequential for cybersecurity automation.

Natural language understanding of security artifacts. LLMs excel at processing the heterogeneous, semi-structured text that permeates security operations: raw log streams, common vulnerabilities and exposures (CVE) and national vulnerability database (NVD) advisories, threat reports, cyber threat intelligence (CTI) feeds, and vulnerability disclosures. Systems such as LogParser-LLM [21] report a grouping accuracy of up to 0.909 on their full 14-dataset benchmark, while LogBatcher [22] processes 3.6 million log messages at a grouping accuracy (GA) of 0.972—the proportion of log messages correctly assigned to their true template group—with a reported 100% reduction in LLM inference cost through batched querying. LogPrécis [23] further demonstrates that an LLM can distil hundreds of thousands of raw log sessions into thousands of structured attack fingerprints.

Code comprehension and generation. Modern code-capable LLMs understand program semantics, data flow, and control flow across multiple languages. This underpins vulnerability detection applications: IRIS [24] exploits LLM code reasoning to infer taint specifications and filter false positives from CodeQL static analysis, achieving whole-repository vulnerability detection that purely rule-based approaches cannot match. Conversely, the same capability introduces risk: LLM-generated code is repeatedly shown to contain CWE-classified security weaknesses across the primary studies cataloged by recent systematic reviews [9], though reported prevalence varies widely by study, model, and benchmark rather than converging on a single rate.

Chain-of-thought and structured reasoning. Chain-of-thought (CoT) prompting [25] substantially improves LLM performance on multi-step security tasks such as attack technique attribution, exploit chain planning, and incident triage: PentestGPT [10] uses a CoT-driven task tree to guide multi-stage penetration test reasoning, explicitly invoking the CoT methodology at each planning step. Structured reasoning in this domain is not limited to CoT, however: CTI-Thinker [26] maps raw threat reports to ATT&CK tactics through a GraphRAG-based reasoning engine and semantic alignment rather than explicit chain-of-thought prompting.

Tool and function calling. LLMs began invoking external tools during generation in 2023, first through self-supervised methods that learn when and how to insert API calls into generated text from a handful of demonstrations [27], and shortly after through vendor-provided function-calling interfaces that constrain tool invocations to a machine-readable, schema-defined format. In security contexts, this allows agents to invoke scanners (Semgrep, Nmap, Metasploit), query knowledge bases (MITRE ATT&CK, NVD), and interact with security information and event management (SIEM) platforms in a programmatically verifiable manner. Function calling is the technical foundation on which all tool-calling and multi-agent architectures described in Section 2.2 are built.

Context window management. Contemporary LLMs support context windows of 128K–1M tokens, enabling in-context processing of entire codebases, extended incident timelines, or multi-turn analyst dialogues without external memory. However, context degradation under long sequences remains a known failure mode [6], observed empirically

in PentestGPT [10] when repeated failed exploit attempts exhaust the model's ability to reconsider prior hypotheses.

Key limitations. Three limitations are particularly salient in security applications. *Hallucination*—the generation of plausible but factually incorrect content—is especially costly when it produces phantom CVEs, incorrect exploit paths, or false threat actor attributions. *Knowledge cutoff* means models may be unaware of recent vulnerabilities or threat campaigns, making retrieval augmentation essential [28]. *Adversarial fragility* exposes LLMs to prompt injection attacks [29], a threat that is amplified when the model operates as an autonomous agent with access to sensitive tools and data.

2.2. Agent Architectures: From Single-Agent to Multi-Agent Systems

An LLM *agent* is distinguished from a bare LLM by the presence of an action loop: a cycle of perceiving context, reasoning over available tools, issuing actions, observing outcomes, and updating state toward a goal [6]. This section defines the six architectural patterns used as the second dimension of our classification framework (Section 3.4); Figure 1 provides a visual overview of all six patterns.

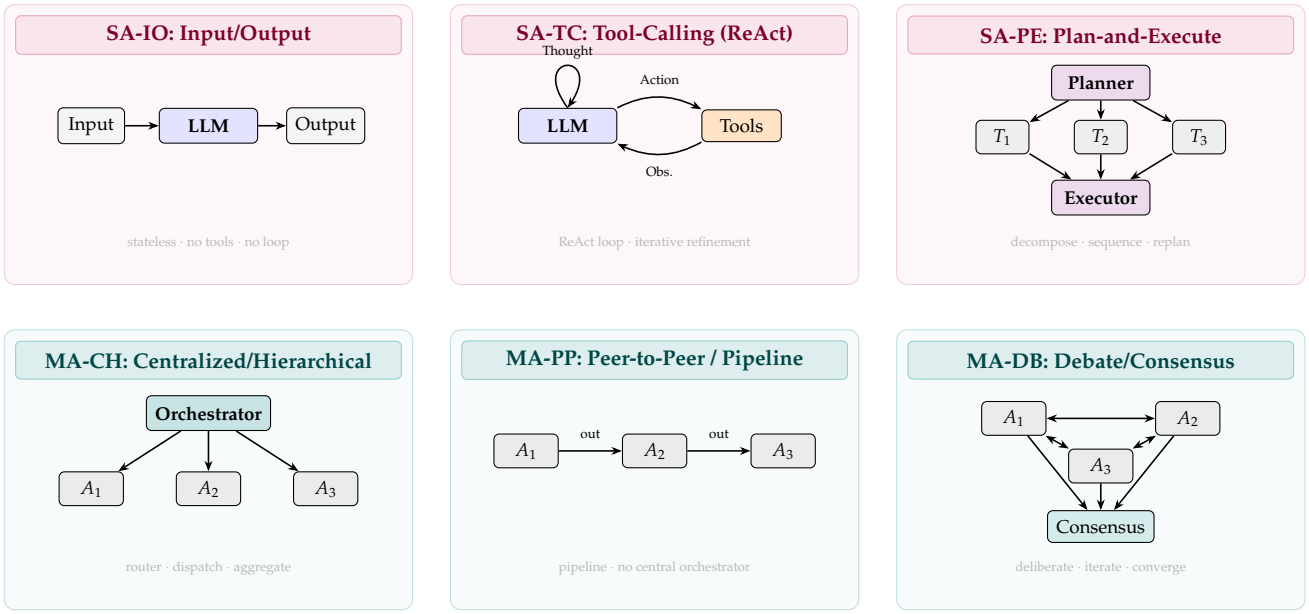


Figure 1. Taxonomy of LLM agent architectural patterns forming the second classification dimension of this review. **Top row (single-agent)**: SA-IO—stateless input/output; SA-TC—iterative ReAct tool-calling loop (Thought → Action → Observation); and SA-PE—plan-and-execute with explicit task decomposition and replanning. **Bottom row (multi-agent)**: MA-CH—centralized hierarchical orchestrator dispatching to specialized subagents; MA-PP—peer-to-peer sequential pipeline with no single controller; and MA-DB—debate/consensus across multiple LLM instances converging to an aggregated output. Arrows denote task dispatch, tool invocation, structured output passing, and deliberation flows.

2.2.1. Single-Agent Architectures

SA-IO (Input/Output). In the simplest pattern, the LLM performs single-shot generation—classification, extraction, or synthesis—and invokes no tools and performs no iterative reasoning of its own. The model receives a structured input (a log line, a code snippet, a CVE description) and produces a structured output (a label, a severity score, a tactic, technique, and procedure (TTP) identifier). It is important to be precise about what qualifies here, because the boundary against the exclusion criteria in Section 3.2 runs through it: SA-IO systems are *embedded in an external automated execution or verification loop* that consumes the model's output and, in most cases, feeds a subsequent generation. Fuz-

zGPT [30] is representative—the model proposes test inputs, an external fuzzing harness executes them, and coverage feedback drives the next round. A system in which the LLM output is consumed by nothing and closes no loop at all is not an agent under our definition and was excluded under EC1. Despite its simplicity, SA-IO yields strong baselines for well-defined subtasks such as ATT&CK TTP extraction from CTI sentences [31].

SA-TC (Single-Agent, Tool-Calling). The ReAct framework [5] formalized the pattern of interleaving reasoning traces (*Thought*) with executable tool invocations (*Action*) and environmental feedback (*Observation*) in a closed loop. SA-TC is the most prevalent architecture in the surveyed corpus, accounting for 30.5% of included studies (18 of 59; Section 3.4). Representative examples include IDS-Agent [32] for explainable intrusion detection and RAGIntel [28] for CTI-augmented attack investigation.

SA-PE (Single-Agent, Plan-and-Execute). The agent explicitly decomposes a high-level task into a sequence of subtasks before execution begins, allowing for structured backtracking and replanning. CTFAgent [33] is representative: it maintains an explicit task tree recording subtasks, dependencies, intermediate artifacts, and strategy revisions, rather than selecting the next action reactively. PentestGPT v2 [34] applies the same pattern to penetration testing, with an Exploration-Guided Adaptive Task-Solving planner that reduces stuck/looping failures by enriching the agent's tool-use strategy with difficulty-aware planning.

2.2.2. Multi-Agent Architectures

MA-CH (Multi-Agent, Centralized/Hierarchical). A central orchestrator—often called a router or coordinator agent—decomposes incoming tasks, dispatches them to specialized subagents, and aggregates their outputs into a unified response [35]. This corresponds to the "supervisor" or "manager–specialist" pattern commonly described in multi-agent orchestration, where a top-level agent performs triage and routing while domain agents (for example, threat intelligence, vulnerability analysis, incident response) run in parallel under shared state and policy constraints. MA-CH provides strong separation of concerns and an explicit audit trail, making it well-suited to high-stakes, regulated environments such as SecOps and financial services. CORTEX [7] exemplifies the pattern: a persistent Orchestrator Agent manages a fixed four-stage pipeline (Orchestration, Behavior Analysis, Evidence Acquisition, and Reasoning and Coordination), enforcing modularity and ensuring coherent handoffs between roles rather than letting the stages hand off unsupervised.

MA-PP (Multi-Agent, Peer-to-Peer/Pipeline). Specialized agents collaborate without a single controlling orchestrator, typically by passing structured outputs along a predefined or dynamically negotiated pipeline. VulnBot [36] implements this pattern with five pipeline agents (Planner, Memory Retriever, Generator, Executor, Summarizer) coordinated through a Penetration Task Graph rather than a designated controller. MyAntFarm.ai [37] demonstrates an extreme case in which a pipeline of agents running on TinyLlama-1B achieves a 100% actionable rate for incident response, against 1.7% for a single-agent deployment of the same model—a difference of nearly two orders of magnitude, on a single system and a single task.

MA-DB (Multi-Agent, Debate/Consensus). Multiple LLM instances deliberate over the same problem, with disagreement-driven iteration converging to a more reliable output [38]. Audit-LLM [39] applies this pattern to insider threat detection, cutting its false-positive rate from over 30% (a no-agent baseline) to under 7% across three evaluation datasets. PhishDebate [40] demonstrates similar robustness gains in phishing website detection by running four heterogeneous models (Qwen2.5-VL-72B-Instruct, Gemini-2.0-Flash, GPT-4o, GPT-4o Mini) as debating agents.

2.2.3. Memory and Knowledge Augmentation

Agent memory is typically classified into *short-term* (in-context working memory, cleared between sessions) and *long-term* memory implemented via a persistent vector store that survives across interactions [6]. In cybersecurity applications, long-term memory stores threat intelligence, incident histories, and domain knowledge (MITRE ATT&CK, CVE/NVD, CAPEC), retrieved at inference time via Retrieval-Augmented Generation (RAG) [41]. In the surveyed corpus, however, in-context learning remains the most common knowledge augmentation strategy at 54.2% of studies, with RAG second at 28.8% (Section 3.4).

2.2.4. Frameworks

Prevalent implementation frameworks include LangChain and LangGraph [42] (graph-based stateful agent orchestration, dominant in production deployments), Microsoft AutoGen [43] (conversational multi-agent coordination, used in the Audit-LLM and multi-agent IR studies), and CrewAI (role-based crew abstraction). All three frameworks support the taxonomy patterns above; the choice among them affects communication protocol, state management, and tool integration mechanics but not the architectural pattern itself.

2.3. Cybersecurity Domain Taxonomy

To provide a consistent analytical frame across heterogeneous literature, this review adopts an eight-function security domain taxonomy aligned to the NIST Cybersecurity Framework (CSF) [44] functions (Identify, Protect, Detect, Respond, Recover) and the MITRE ATT&CK enterprise lifecycle [45]. Table 2 defines each function and its primary operational scope.

Table 2. Security domain taxonomy used for paper classification in this review. Each code is used consistently in the master classification table (Appendix B).

Code	Security Function	Scope and Primary Tasks
TI	Threat Intelligence and CTI	Collection, extraction, and structuring of threat data; knowledge graph construction; TTP identification; threat actor attribution; and indicator of compromise (IoC) enrichment aligned to MITRE ATT&CK.
VA	Vulnerability Assessment	Detection, localization, severity estimation, and remediation guidance for software vulnerabilities in source code or running systems; and CVE/CWE classification.
DS	DevSecOps and SOC Automation	Security integration into continuous integration and continuous delivery (CI/CD) pipelines; static application security testing (SAST) and software composition analysis (SCA); secure code generation; infrastructure-as-code (IaC) security; supply-chain risk; and SOC workflow automation and alert triage.
ID	Intrusion Detection	Anomaly and signature-based detection of malicious activity in network traffic or host logs; insider threat detection; and detection explainability.

Table 2. *Cont.*

Code	Security Function	Scope and Primary Tasks
PT	Penetration Testing and Red Teaming	Controlled adversarial simulation: reconnaissance, vulnerability analysis, exploitation, lateral movement, and post-exploitation reporting.
IR	Incident Response and Forensics	Detection-to-containment workflow automation; evidence collection and timeline reconstruction; playbook execution; and post-incident analysis.
CR	Compliance and Risk Assessment	Mapping of system configurations, threat models, and controls to regulatory frameworks (DORA, NIS2, PCI-DSS, NIST 800-53, ISO/IEC 27001 [46]); and risk scoring.
PH	Phishing and Social Engineering	Detection of phishing emails, URLs, and websites; generation of phishing awareness content; and defense against LLM-generated social engineering at scale.

The eight functions are not mutually exclusive: several systems (e.g., CyberRAG [47] spanning DS and ID; ThreatModeling-LLM [48] spanning VA and CR) address more than one domain simultaneously. For such papers, the primary function is assigned based on the authors' stated objective, with secondary functions noted in the classification table. This taxonomy also defines the structure of Section 4, where each subsection is dedicated to one function.

3. Methodology

This review was conducted and is reported in accordance with the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) 2020 statement [49]. The completed PRISMA 2020 checklist, mapping every reporting item to its location in this manuscript, is provided as Table S1 in the Supplementary Materials.

The review protocol covers the search strategy, source selection, screening criteria, and the multi-dimensional classification framework, and is published in full with the data (see the Data Availability Statement). It was **not** registered prospectively with PROSPERO, OSF, or any other registry, and the protocol document was written up as a record of the procedure followed rather than as a pre-commitment; readers should weight it accordingly.

3.1. Search Strategy

Retrieval procedure. Records were identified through *web-search-mediated retrieval* rather than through queries issued directly to individual database interfaces. Fifty-nine structured queries were executed against a general web search engine, combining quoted phrases, Boolean `OR`alternation, named-system lookups, and `site:`-restricted operators targeting scholarly hosts. Only the first page of results was assessed for each query, a standard search-engine pagination cap rather than a limit on the underlying result set; the complete set of queries is reproduced verbatim in Appendix C.

We adopted this procedure deliberately, and it carries a trade-off that readers should weigh. LLM agent research in cybersecurity is overwhelmingly disseminated as preprints and moves faster than the indexing cycles of subscription databases; a search engine surfaces cross-venue and recently posted work that a per-database query set tends to miss, and it retrieves the same record through multiple independent routes, which is useful

evidence of salience. The cost is that coverage is determined by an opaque ranking function rather than by a declared index, and that recall cannot be bounded. We therefore make no claim to exhaustive coverage of any particular database. Section 8 states this as a limitation, and Appendix C publishes the full query record so that the retrieval can be repeated and its coverage independently assessed.

Sources represented. The retrieved records resolve predominantly to **arXiv** (categories cs.CR, cs.AI, and cs.SE), which hosts the majority of recent LLM agent research prior to formal publication, followed by **Semantic Scholar**, **ACM Digital Library**, **IEEE Xplore**, **USENIX**, **MDPI**, **SpringerLink**, and **Elsevier ScienceDirect**. Appendix C reports the distribution of retrieved records across these hosts. Because retrieval was not partitioned by database, the source distribution is descriptive of what was found rather than a statement of coverage per index.

Characterizing coverage. A retrieval strategy of this kind cannot bound its own recall analytically, so we measured it. We executed an equivalent Boolean strategy directly against six bibliographic databases—the ACM Digital Library, IEEE Xplore, Scopus, Elsevier ScienceDirect, SpringerLink, and arXiv—returning 2682 records, 2331 unique. The queries, execution dates, and per-database yields are given in Appendix D. This exercise characterizes the corpus; it is not a second identification stage, and no study entered the corpus on the strength of appearing in it.

Two figures come out of it, and they should be read together.

Recovery. The database searches return 44 of the 51 included studies that the query set had surfaced (86.3%), and 47 of all 59 (79.7%); the eight studies contributed by citation chaining are not expected to appear in any query and largely do not. The seven unrecovered studies cluster by venue rather than by topic: one is published at USENIX Security and one at a small regional conference, neither indexed by any source searched here, and five are arXiv preprints outside the reach of the specific strings used. Coverage of this literature is therefore governed by which indexes one consults, not by any systematic blind spot in the retrieval itself. Sensitivity analysis supports the same reading: removing arXiv from the verification set collapses recovery to 36.5%, whereas removing the ACM Digital Library or ScienceDirect changes it not at all, which is a direct measure of how strongly this field still lives in preprints.

Scale. The same searches returned 2283 records not in the corpus, 828 of which mention both an agent or LLM term and a security term. A random sample of 60 of those 828, screened against the criteria in Section 3.2, was 41.7% eligible (95% CI 29–54%), implying on the order of 345 further eligible studies (CI 242–448) within these six databases alone. We report this because the recovery figure alone would invite a stronger conclusion than the evidence supports. The corpus is internally complete and fully coded, and it recovers well against an independent search—but it is a documented subset of a literature we estimate at roughly 400 eligible studies, not a census of it. Section 1 states the consequences for what this review claims; Section 8 records it as a limitation.

Date range. January 2022–April 2026. The lower bound was chosen to coincide with the emergence of instruction-tuned, tool-capable LLMs (ChatGPT/GPT-4 era); earlier work on neural network-based security tools does not satisfy the agent definition established in Section 2.2.

Keyword clusters. Queries were derived from four complementary concept clusters. The clusters below describe the conceptual coverage; they are not themselves the executed strings, which are given verbatim in Appendix C. Terms were combined with `OR` within a cluster and, where a query targeted an intersection, with implicit conjunction across clusters:

- *Cluster 1—Core agent and security:* “LLM agent cybersecurity”; “large language model agent security”; “AI agent cyber defense”; “multi-agent LLM security”; and “autonomous agent cybersecurity”.

- *Cluster 2—Security functions:* "LLM penetration testing"; "LLM vulnerability detection"; "LLM threat intelligence"; "LLM intrusion detection"; "LLM incident response"; "LLM SOC automation"; "LLM malware analysis"; "LLM DevSecOps"; "LLM code vulnerability"; and "LLM compliance security".
- *Cluster 3—Agent architecture:* "ReAct agent security"; "multi-agent orchestration cybersecurity"; "LangGraph security"; "AutoGen cybersecurity"; "tool-calling agent security"; "RAG cybersecurity agent".
- *Cluster 4—Financial sector and regulation:* "LLM financial cybersecurity"; "AI agent DORA compliance"; "LLM fintech security"; and "LLM SIEM financial".

Forward and backward citation chasing was performed on all included studies and on six contextual survey papers (e.g., [11,12]) to recover relevant works not surfaced by keyword search.

3.2. Inclusion and Exclusion Criteria

Inclusion criteria. A study was included if it satisfied *all* the following:

1. *Agentic LLM contribution (IC1)*: The paper proposes, implements, or empirically evaluates an LLM-based agent—defined as an LLM operating within an action loop involving at least one of the following—tool invocation, multi-turn iterative reasoning, planning, inter-agent communication, or embedding in an external automated execution or verification loop that consumes the model's output and conditions a subsequent generation step—applied to a cybersecurity task. This last condition is what the single-shot SA-IO pattern (Section 2.2) must satisfy to qualify; it is stated as the corresponding carve-out in EC1 below.
2. *Publication date (IC2)*: January 2022–April 2026.
3. *Source quality (IC3)*: Published at a peer-reviewed venue (conference or journal), OR as an arXiv preprint with five or more independent citations or from a demonstrably active research group.
4. *Language (IC4)*: Full text available in English.

Exclusion criteria. A study was excluded if it met *any* of the following:

1. *No agentic loop (EC1)*: The LLM is used as a zero-shot or few-shot classifier, extractor, or code generator whose output closes no loop—no iterative tool use, no multi-turn reasoning, no planning component, no inter-agent communication, *and* no external execution or verification stage that consumes the output and conditions subsequent generation—even if the paper applies to a security task. The final clause mirrors the corresponding condition added to IC1 above and is the operative one for the SA-IO systems retained in the corpus (Section 2.2): a model that emits a label consumed only by a human reader is excluded, whereas one whose output is executed by an automated harness that feeds results back is a single-shot component of an agentic system and is retained.
2. *Evaluation infrastructure (EC1b)*: Benchmarks, task suites, and evaluation harnesses are cataloged as evaluation infrastructure and discussed in Section 6.3, but are not coded as primary studies unless the work also contributes an agent system evaluated against that infrastructure.
3. *Security of LLMs only (EC2)*: The paper addresses attacks against LLMs (adversarial examples, jailbreaks, backdoors) without also contributing an LLM-based agent for defensive security tasks.
4. *Insufficient empirical contribution (EC3)*: Position papers, opinion pieces, or purely architectural proposals with no experimental evaluation.

5. *Duplicate or superseded (EC4)*: When multiple versions of the same system exist (e.g., preprint and published paper, or v1/v2 on arXiv), only the most recent substantive version was retained.
6. *Out of scope (EC5)*: Pre-2022 works; non-English full texts; and works addressing cybersecurity only as a secondary or illustrative application of a general agent framework.

Title and abstract screening and full-text eligibility assessment were performed by the lead author applying the criteria defined above. To improve screening consistency, a systematic AI-assisted pass was conducted in parallel using Anthropic's Claude (model: Claude 3.5 Sonnet, accessed April 2026) prompted with the explicit inclusion and exclusion criteria. The tool was given each record's title and abstract, and, where a full-text decision was required, the relevant sections of the paper; it returned a proposed decision, the criterion invoked, and a supporting quotation. Its use was confined to the screening phase. It was not used to retrieve records, to code the five taxonomy dimensions, to compute any statistic reported here, or to generate any analysis, argument or conclusion in this paper. It functioned as a consistency check rather than as an independent screener, and no decision entered the review on its output alone; the division of authorial responsibility is set out immediately below. This hybrid human–AI screening protocol follows emerging practice in AI-assisted systematic reviews [50,51].

How the deposited screening record was produced. The per-record screening file published with this paper (see the Data Availability Statement) was compiled after the initial submission, and we describe its provenance precisely because the two are not the same exercise. The *inclusion* decisions are the authors': the original corpus was assembled by the lead author, and every study added at revision was individually adjudicated by the lead author against the criteria above, with the reasoning recorded per study in the deposit. The *exclusion* codings for the remaining records were produced by an AI-assisted pass over the retrieval record applying the same published criteria, with a supporting evidence note and source link for every record, and were then adopted by the authors, who take responsibility for them. They were not individually re-verified by a human reader. We consider this proportionate—the inclusion boundary determines the corpus and received author judgment, while the exclusion codings are a reporting artifact that a reader can audit directly against the deposited evidence—but the distinction is material and we prefer to state it. Twenty-nine records identified by citation chaining were assessed at full text and excluded; their individual identities and reasons were not retained in the authors' working notes and are recorded as such rather than reconstructed.

Three limitations of this protocol should be stated directly, and are returned to in Section 8. Screening was performed by a single reviewer; no second reviewer independently re-screened a sample, so no inter-rater agreement statistic is reported. The review protocol was not registered prospectively with PROSPERO or OSF. And no formal risk-of-bias instrument was applied beyond the E1–E4 evaluation rigor coding, which assesses the strength of each study's evaluation rather than the risk of bias in its conduct. In place of an agreement statistic, we publish the complete per-record screening decision set with the criterion applied to each, which allows a reader to audit the screening directly rather than infer its reliability from a coefficient.

3.3. Study Selection and PRISMA Flow

The 59 queries returned 584 results in total, of which 520 carried an identifiable bibliographic host (the remainder were aggregator pages, search refinements, or results without a resolvable source). Title-level deduplication—which is substantial here, because the retrieval strategy deliberately surfaces the same work through multiple query routes—reduced these to 380 unique records. Removing non-scholarly items (vendor blog posts, code repositories, trade press, and analyst notes, which the search engine returns

alongside research output and which no database interface would have surfaced) left 174 unique scholarly records. A further 37 records were identified by forward and backward citation chaining from included studies and from six contextual survey papers, giving **211 unique scholarly records** entering title and abstract screening.

Screening the 174 search-derived records against the criteria in Section 3.2 identified 52 studies meeting all inclusion criteria, together with 21 further records that proved on inspection to be duplicate retrievals of those same studies under truncated or otherwise mangled titles (EC4); the remaining 101 were excluded. All 37 records added by citation chaining passed this initial title/abstract screening and proceeded to full-text review. In total, **122 records were excluded at title and abstract screening**, predominantly records lacking an LLM or agent component—Cluster 1 mis-hits retrieving classical machine-learning security tools—along with the duplicate retrievals just described and works falling outside cybersecurity scope.

Eighty-nine records were retrieved for full-text review, of which 30 were excluded for the reasons presented in Section 3.2: papers contributing no agentic loop (EC1), contributing only a benchmark or evaluation study (EC1b), addressing the security of LLMs without a defensive agent contribution (EC2), offering no empirical evaluation (EC3), or falling outside scope (EC5). The final corpus comprises **59 primary studies**. Six additional survey papers were cataloged as contextual references but are excluded from the classification table because they do not contribute primary empirical results [1,11–14,29].

Two features of this flow warrant comment. First, 51 of the 59 included studies were surfaced by the query set and 8 only by citation chaining, so the supplementary strategy contributed 13.6% of the final corpus—chaining was not a formality. Second, the deposited screening record (see the Data Availability statement) reflects a re-screening of the retrieval record against the stated criteria, conducted after initial submission: 16 studies that met the criteria but had not been carried into the original corpus were added, and one study whose framing as an LLM-based agent is not borne out by its method (Section 4.4) was removed.

Figure 2 presents the PRISMA 2020 flow diagram.

3.4. Classification Framework

Each included study was independently coded along five dimensions. The dimensions and their admissible values are formally defined in Table 3; the resulting master classification table for all 59 studies is provided in Appendix B.

Table 4 summarizes the distribution of the 59 included studies across these five dimensions, providing a quantitative overview of the field's current state that informs the domain-by-domain analysis in Section 4 and the gap analysis in Section 7.

Several observations from Table 4 merit immediate attention. Penetration testing (PT) accounts for 50.8% of the corpus—an outright majority—reflecting the maturity and tractability of that domain and the tractability of its success criteria. Compliance and risk assessment (CR) is represented by a single paper—a critical gap in the context of escalating regulatory requirements such as DORA [15] and the EU AI Act [17]. At the architectural level, MA-CH—the pattern most suited to enterprise orchestration—appears in 18.6% of works, despite its theoretical advantages for complex, multi-domain security workflows. This is worth stating carefully, because the two facts are easily conflated: MA-CH produces the largest reported effect sizes in the corpus while resting on the smallest evidence base within the multi-agent family. Prevalence and performance point in opposite directions, and Section 5 treats them separately. Most strikingly, *no included study reports a production deployment (E4)*, meaning the entire field of LLM-based cybersecurity agents currently rests on controlled lab evaluations. These patterns structure the gap analysis in Section 7. Figure 3 shows how the corpus is distributed over time: it concentrates in a 24-month window, so the patterns above describe a literature that is both recent and compressed.

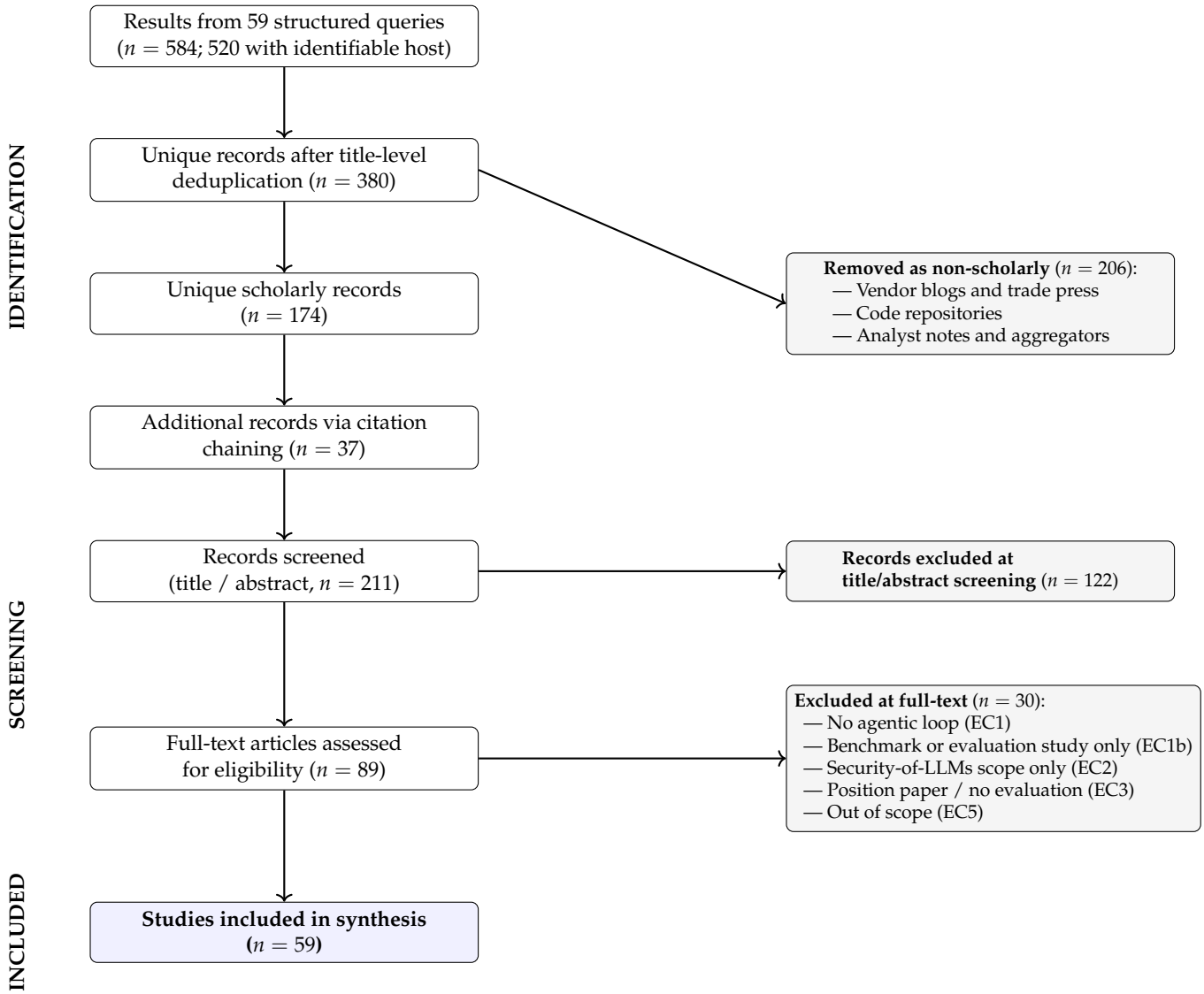


Figure 2. PRISMA 2020 flow diagram for the systematic literature review. Counts reflect records at each stage of identification, screening, eligibility assessment, and final inclusion. Vertical arrows carry records forward through the review; horizontal arrows are unidirectional by construction and denote records leaving the review at that stage. No records re-enter the flow once excluded. Following PRISMA 2020, exclusion reasons are itemized at the full-text eligibility stage; title and abstract screening reports the total only. EC1–EC5 labels refer to the exclusion criteria defined in Section 3.2. The full query record underlying the identification stage is given in Appendix C.

Table 3. Five-dimensional classification framework. Each included study receives one or more values per dimension; multi-function papers receive a primary code and secondary codes noted in parentheses.

Dimension	Code(s)	Definition
D1: Security Function	TI	Threat Intelligence and Cyber Threat Intelligence
	VA	Vulnerability Assessment
	DS	DevSecOps and Secure Code Analysis
	ID	Intrusion Detection
	PT	Penetration Testing and Red Teaming
	IR	Incident Response and Digital Forensics
	CR	Compliance and Risk Assessment
	PH	Phishing and Social Engineering Detection

Table 3. *Cont.*

Dimension	Code(s)	Definition
D2: Agent Architecture	SA-IO	Single-Agent, Input/Output: LLM as stateless classifier or extractor; no iterative loop, and no tool use.
	SA-TC	Single-Agent, Tool-Calling: ReAct-style loop with external tool invocation and observation feedback.
	SA-PE	Single-Agent, Plan-and-Execute: Explicit task decomposition into a subtask plan before execution begins.
	MA-CH	Multi-Agent, Centralized/Hierarchical: Dedicated orchestrator dispatches tasks to specialized subagents; and aggregates results.
	MA-PP	Multi-Agent, Peer-to-Peer/Pipeline: Agents collaborate via a structured pipeline without a single central controller.
	MA-DB	Multi-Agent, Debate/Consensus: Multiple agent instances deliberate iteratively to converge on a final output.
D3: Knowledge Augmentation	ICL	In-Context Learning: Prompting only (zero-shot, few-shot, chain-of-thought); no external retrieval or fine-tuning.
	RAG	Retrieval-Augmented Generation: External knowledge base (vector store, graph DB) queried at inference time.
	FT	Fine-Tuned: Domain-adapted model weights (full or parameter-efficient fine-tuning, e.g., LoRA).
	HYB	Hybrid: Combination of two or more of ICL, RAG, and FT.
D4: HITL Posture	FA	Fully Automated: No human involvement in the operational loop.
	HA	Human-Approval: Human approves specific high-stakes actions before execution.
	HG	Human-Guided: Human provides periodic input or direction; agent executes autonomously between interactions.
	HO	Human-Overseen: Human reviews all outputs before they take effect; agent does not execute directly.
D5: Evaluation Rigor	E1	Qualitative/Conceptual: No quantitative evaluation; design or architecture paper only.
	E2	Public Benchmark: Evaluation on a publicly available, standardized dataset or benchmark suite.
	E3	Controlled Experiment/Test Bed: Custom experimental environment, lab-constructed scenarios, or user studies.
	E4	Real-World Deployment: Evaluation in a live production environment with real security events or vulnerabilities.

Table 4. Distribution of the 59 included studies across the five classification dimensions. Percentages are rounded to one decimal place. Every dimension reports the *primary* code only, so each column sums to exactly 59 and each percentage column to 100%. Six studies carry a secondary security function (CyberRAG and Novel Defense: intrusion detection; Sahay et al. [52]: DevSecOps; AutoPentester and ThreatModeling-LLM: vulnerability assessment; and Tellache et al. [53]: threat intelligence); these are recorded in Appendix B but are not counted here.

Dim.	Code	*n*	%
D1: Security Function	PT	30	50.8
	TI	8	13.6
	VA	6	10.2
	IR	5	8.5
	DS	3	5.1
	PH	3	5.1
	ID	3	5.1
	CR	1	1.7

Table 4. *Cont.*

Dim.	Code	*n*	%
D2: Architecture	SA-TC	18	30.5
	SA-IO	16	27.1
	MA-CH	11	18.6
	MA-PP	8	13.6
	SA-PE	3	5.1
	MA-DB	3	5.1
D3: Knowledge Aug.	ICL	32	54.2
	RAG	17	28.8
	HYB	7	11.9
	FT	3	5.1
D4: HITL Posture	FA	52	88.1
	HG	6	10.2
	HA	1	1.7
	HO	0	0.0
D5: Eval. Rigor	E3	44	74.6
	E2	14	23.7
	E1	1	1.7
	E4	0	0.0

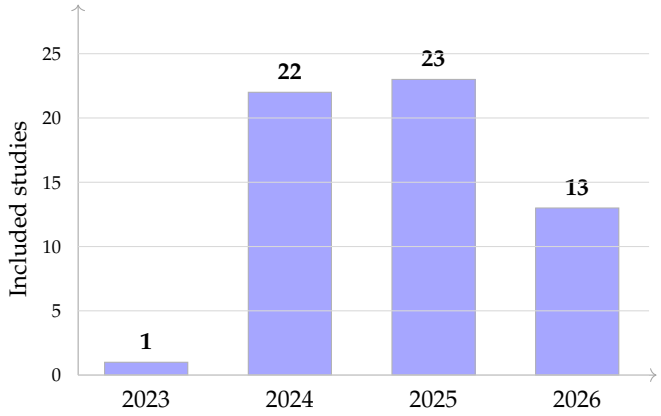


Figure 3. Publication year distribution of the 59 included studies. The counts are by the year of the version cited, which for several studies retrieved as preprints inside the search window is the year of subsequent formal publication (see Appendix B). Two features are relevant to interpreting the review: the corpus is concentrated in a 24-month window, and the single 2023 entry marks the practical start of this literature rather than an artifact of the search window, which opens in January 2022.

4. LLM Agents by Security Domain

This section analyzes the 59 included studies organized by primary security function. For each domain, we describe the challenge landscape, survey representative works mapped to the classification framework of Section 3.4, report key quantitative results, and identify domain-specific limitations. Table A1 in Appendix B provides the full per-paper classification. For conciseness, each citation is referred to by the system name used by the authors.

4.1. Threat Intelligence and Cyber Threat Intelligence (TI)

Domain challenges. Cyber Threat Intelligence (CTI) analysts face a deluge of heterogeneous artifacts—threat reports, blog posts, CVE advisories, dark-web forum extracts—that must be parsed, enriched, and mapped to structured frameworks such as MITRE ATT&CK before they become actionable. Manual processing is slow and does

not scale; existing NLP-based approaches are brittle to the linguistic diversity of threat reports and require constant retraining as adversary TTPs evolve. LLM agents offer a compelling alternative: they can extract structured triples from free-form text, infer ATT&CK technique identifiers, and attribute threat actors across corpora in a unified reasoning loop [31,54].

Representative works. Seven of the eight TI papers in the corpus span knowledge graph construction, real-time feed integration, and TTP recognition from binary artifacts; the eighth, discussed separately below, targets the schema-drift problem shared by all of them.

Knowledge graph construction. CTINexus [55] (SA-IO, ICL) constructs cybersecurity knowledge graphs from 150 real-world CTI reports using optimized in-context learning with hierarchical entity alignment. CTIKG [54] extends this to a multi-agent design (MA-PP) with a dual short-term/long-term memory architecture, achieving 86.88% precision in KG construction—at least 30% above the state-of-the-art classical baselines. CTI-Thinker [26] further integrates GraphRAG for structured ATT&CK alignment, enabling automated reasoning over advanced persistent threat (APT) attack chains with measurable improvements in attribution precision.

CVE-to-ATT&CK mapping. Crimson [56] (SA-IO, HYB) demonstrates that a A 7B-parameter Llama2 model fine-tuned with Retrieval-Aware Training can achieve near-GPT-4 performance on CVE-to-ATT&CK mapping while producing substantially fewer hallucinations, making it viable for privacy-sensitive, on-premises financial deployments.

Real-time intelligence integration. Proactive Threat Intelligence [57] (SA-IO, RAG) connects live CVE, CWE, EPSS, and CISA KEV feeds via a Milvus vector database, enabling dynamic threat reasoning that reflects the current vulnerability landscape. Tseng et al. [58] (SA-IO, RAG) process more than 50 CTI reports through an eight-step pipeline that extracts over 2900 candidate IOCs (2300 validated against manual ground truth), builds a relationship graph between them, and generates RegEx patterns to help SOC analysts assemble SIEM correlation rules, narrowing—though not fully closing—the gap between intelligence collection and detection deployment.

Binary-level TTP recognition. TTPDetect [59] (SA-TC, RAG) operates at the function level on decompiled malware binaries, achieving 93.25% precision in MITRE ATT&CK TTP identification—a task previously requiring deep reverse engineering expertise.

Key results. Crimson's Retrieval-Aware Training demonstrates that domain fine-tuning is a viable alternative to frontier-model API dependence for CTI tasks. CTIBench [31] and AthenaBench [60] together establish a growing evaluation infrastructure, though the two benchmarks use different task formulations, complicating cross-study comparison.

Ontology-guided extraction. OntoLogX [61] addresses a weakness common to the systems above: extracted knowledge graphs are only as reusable as the schema they commit to. The system grounds extraction from raw security logs in an explicit ontology, with retrieval supplying relevant prior graph structure and an iterative correction stage validating the output before it is committed. The authors evaluate across several backbones and then map the resulting entities to MITRE ATT&CK techniques. Ontology grounding is a partial answer to the schema-drift problem, though it trades flexibility for consistency and, like every CTI system in this corpus, is evaluated only on English-language sources.

Limitations. The dominant limitation is *scalability to continuous feeds*: all eight TI papers evaluate on static corpora. Hallucination in low-frequency entity classes (emerging threat actors, novel malware families) remains a shared failure mode. No TI paper addresses multilingual threat reports, despite a significant volume of non-English CTI material in Eastern European and East Asian campaigns.

4.2. Vulnerability Assessment (VA)

Domain challenges. Vulnerability assessment encompasses detection, localization, severity estimation, and patch suggestion across source code, compiled binaries, and running systems. Traditional SAST tools suffer from high false positive rates and poor generalization across programming languages. LLM agents address these limitations through semantic code comprehension, dataflow inference, and iterative tool feedback, enabling whole-repository analysis previously beyond the reach of automated systems.

Representative works. The four VA papers below represent three distinct application sub-areas: protocol fuzzing, dataflow analysis, and SAST augmentation.

Protocol fuzzing. ChatAFL [62] (SA-IO, ICL, evaluated at E3) addresses a fundamental bottleneck in stateful fuzzing: the LLM extracts protocol grammars directly from RFC documents, guiding mutation-based fuzzing without manual specification. Across six real-world protocol implementations spanning five protocols (RTSP, FTP, SIP, SMTP, DAAP), ChatAFL achieves 6.1× faster coverage than its AFLNet baseline and discovers nine new vulnerabilities, including one in a production FTP implementation. FuzzGPT [30] (SA-IO, ICL) takes a complementary approach: few-shot-prompted code LLMs generate historically buggy code patterns to trigger edge cases in deep learning frameworks. Applied to the PyTorch and TensorFlow deep-learning frameworks as evaluated in that study, FuzzGPT finds 76 bugs, of which 49 were previously unknown and 11 are classified as high-priority security vulnerabilities.

Compilation-free dataflow analysis. LLMDFA [63] (SA-IO, ICL) replaces traditional compilation-dependent dataflow analysis with an LLM-driven pipeline paired with a Z3 SMT solver for verification. This enables vulnerability detection without build environments, significantly lowering the barrier for ad hoc code auditing. The authors report average per-task costs of $0.04–$0.14 and total experiment cost of $1622 across their full evaluation suite, but flag frequent prompting's time overhead as better suited to targeted, high-value analysis than continuous whole-codebase scanning.

SAST augmentation. IRIS [24] (SA-IO, ICL, evaluated at ICLR 2025) exemplifies the neuro-symbolic paradigm: the LLM infers taint source and sink specifications from API documentation, which are then supplied to CodeQL for path-sensitive vulnerability detection. This combination achieves whole-repository reasoning that neither the LLM nor CodeQL can accomplish independently.

Key results. The VA domain is distinguished by its breadth of evaluation settings and by the emergence of neuro-symbolic integration as a recurring effective pattern. Backbone choice is notably diverse rather than GPT-4-concentrated—only IRIS uses GPT-4 as its headline model; the rest span GPT-3.5-family, Codex, Kimi-K2, and Llama-family models—and five of the six papers rely on in-context prompting alone, with no VA paper in the corpus using fine-tuning as its headline configuration.

Constraint solving and protocol fuzzing. Two further systems extend the domain in different directions. VulSolver [64] couples static analysis to an LLM that propagates parameter state across caller–callee pairs: SAST path extraction feeds branch-method analysis, context is carried forward, and the model determines the next state at each step. The decomposition is fixed by the framework rather than planned by the model, which places it in SA-IO rather than SA-PE. MultiFuzz [65] (MA-PP, RAG) applies specialized agent crews to network protocol fuzzing, with a shared dense retrieval service over RFC text supplying grammar extraction and seed enrichment, and a dedicated crew invoked when coverage plateaus. It is benchmarked directly against ChatAFL, one of the earlier systems in this same domain—a rare instance of like-for-like comparison in a corpus otherwise characterized by bespoke evaluation.

Limitations. The near-complete absence of DAST (dynamic application security testing) agents is notable; all VA papers focus on static analysis or fuzzing, leaving runtime vulnerability assessment largely unaddressed. LLMDFA's cost profile renders it impractical for continuous integration, and no VA paper evaluates against production CI/CD pipelines.

4.3. DevSecOps and SOC Automation (DS)

Domain challenges. SOC analysts are routinely exposed to thousands of security information and event management (SIEM) and extended detection and response (XDR) alerts per day, with manual alert triage typically taking minutes to complete. Adversary dwell time—the interval between initial compromise and detection—has fallen from a global median of 205 days in 2014 to 11 days in 2024, though it remains material and is longer in organizations that detect intrusions through external notification (26 days) than internally (10 days) [66]. Simultaneously, DevSecOps practitioners must integrate security gates into CI/CD pipelines without creating velocity-killing false-positive noise. LLM agents address both challenges by enabling semantic alert correlation, policy-driven automation, and natural-language-to-detection-rule generation.

Representative works. Three papers in the corpus have DevSecOps/SOC automation as their primary function. A fourth, Sahay et al. [52], carries DS as a secondary function; its quantitative evaluation targets an intrusion-detection decision, so it is discussed as an ID-primary study in Section 4.4, with a cross-reference here because its SIEM-integration framing is squarely a DS concern.

Multi-agent alert triage. CORTEX [7] (MA-CH, RAG) implements a centralized multi-agent architecture mirroring SOC team structure: a named Orchestrator Agent manages a fixed four-stage pipeline, enforcing modularity and coherent handoffs as it routes each alert through a Behavior Analysis Agent, workflow-specific Evidence Acquisition Agents with typed tool calls, and a Reasoning and Coordination Agent that finalizes the verdict. Evaluated on real-world enterprise SOC investigation traces spanning ten alert workflows—including O365, AWS, Salesforce, and endpoint scenarios—CORTEX produces auditable investigation briefs at a median of 152.4 s per ticket. The trade-off is significant: CORTEX is 3.4× slower and consumes 5.68× more tokens than a single-agent baseline, positioning it as a tier-2 escalation tool rather than a first-pass filter.

This trade-off is worth making concrete, because "5.68× more tokens" does not by itself tell an operations lead whether to deploy the system. Consider a Tier-1 queue and let c be the per-alert inference cost of a single-agent baseline, w the fully loaded analyst cost per minute, and m the analyst-minutes a correct automated investigation brief displaces. Multi-agent triage is cost-favorable when $4.68\,c < w\,m\,\rho$, where ρ is the proportion of briefs accurate enough to be acted on without rework. With inference costs for a single alert investigation currently one to two orders of magnitude below a loaded analyst-hour, the inequality is satisfied for surprisingly small values of m—the binding constraint is ρ, not token spend. In other words, the token overhead of orchestration is close to irrelevant against labor cost; what determines viability is whether the extra agents actually raise brief quality enough to remove the analyst from the loop, and the 3.4× latency penalty is what rules the approach out for real-time first-pass filtering.

We present this as an illustrative model rather than a measurement. No study in the corpus reports production cost data, deployment-scale throughput, or rework rates, so ρ in particular is unconstrained by published evidence. That absence is itself notable and belongs alongside the E4 evaluation gap discussed in Section 6.3: the field has not yet produced the operational figures a security organization would need to justify adoption.

CyberRAG [47] (SA-TC, HYB) takes a modular approach: an agentic RAG system with iterative multi-phase retrieval and per-family fine-tuned classifiers produces SOC-ready

structured reports, achieving production-quality output on a multi-family attack classification task spanning SQL injection, XSS, and SSTI; its secondary intrusion-classification function is discussed alongside the ID-primary studies in Section 4.4.

Key results. The three DS-primary papers span three distinct architectural patterns: CORTEX's centralized orchestration (MA-CH), CyberRAG's model-driven iterative retrieval (SA-TC), and the CodeQL repair study's single-pass generation inside an external verification loop (SA-IO, discussed below). Across CORTEX and CyberRAG, a pattern visible elsewhere in the corpus holds: multi-agent and tool-calling designs outperform single-pass generation on precision-sensitive SOC tasks, at a relative latency and token cost that only CORTEX quantifies—CyberRAG reports no comparable overhead figures.

Secure code generation with automated repair feedback. Yan et al. [67] study whether static-analysis feedback can close the loop on LLM-generated code. Generated implementations are analyzed by CodeQL, the resulting findings are returned to the model, and a revised implementation is produced; a second configuration has GPT-4o convert raw CodeQL output into diagnosis and repair guidance before it is passed back. The post hoc repair configuration is a clear instance of the SA-IO pattern under the definition in Section 2.2—single-shot generation inside an external verification loop—and it is instructive that the loop closes through a deterministic analyzer rather than through the model's own judgment, which is precisely the grounding that Section 6.1 argues is missing elsewhere.

Limitations. All three DS-primary papers are evaluated in controlled environments on historical or synthetic datasets; no live SOC integration study exists among them. (Sahay et al.'s [52] Splunk-integrated system, discussed in Section 4.4, is the corpus's closest approach to production SIEM integration but is Splunk-specific, limiting generalization to other platforms.) Production-scale cost and throughput data—dollar cost per ticket, deployment-scale rate limits, and rework frequency—is absent from all three DS papers, even where relative latency and token overhead are reported (CORTEX); this leaves a gap between the reported relative figures and operational viability at SOC scale.

4.4. Intrusion Detection (ID)

Domain challenges. Anomaly-based intrusion detection requires real-time classification of network flows or host events against a dynamic baseline, with the additional requirement of producing human-interpretable explanations to satisfy analyst oversight mandates. Classical ML-based IDS systems produce black-box decisions that are difficult to audit; LLM agents introduce the possibility of detection with natural-language justification and adaptive sensitivity.

Representative works. Three studies in the corpus have intrusion detection as their primary function, and a fourth (CyberRAG, coded primarily as DevSecOps) carries intrusion detection as a secondary function. All four demonstrate proof-of-concept viability rather than production readiness.

IDS-Agent [32] (SA-TC, RAG) is the first LLM-powered IDS with explainable detection outputs, evaluated on the ACI-IoT and CIC-IoT benchmarks. It achieves F1 scores of 0.97 and 0.75, respectively, while generating natural-language explanations for each detection decision and supporting customizable sensitivity thresholds—enabling operators to tune the precision/recall trade-off without retraining. Critically, IDS-Agent demonstrates zero-day adaptation: when its underlying classifiers disagree and confidence falls below a threshold, the system flags the sample as an unknown attack rather than forcing a classification, achieving a 0.61 top-1 recall on unseen attack types (versus 0.41 and 0.47 for two dedicated zero-day baselines)—a capability absent from fixed-signature IDS systems.

Audit-LLM [39] (MA-DB, ICL) targets insider threat detection with a debate paradigm: a Decomposer, Tool Builder, and Executor agent assemble a case from log evidence, and multiple LLM instances then deliberate over it through an Evidence-based Multi-Agent Debate (EMAD) mechanism before reaching a verdict. Evaluated on three public insider-threat datasets (including CERT and PicoDomain), Audit-LLM's full pipeline cuts its false-positive rate from over 30% (a no-agent GPT-3.5 baseline) to under 7% across all three evaluation datasets, and beats the best external insider-threat baseline's FPR by 1.5–2.8 percentage points—a result with direct operational value in environments where analyst fatigue from false alerts is a documented organizational risk.

Sahay et al.'s policy-guided threat hunting framework [52] (MA-CH, ICL) chains an autoencoder anomaly detector, a deep reinforcement learning triage module, and an LLM orchestrator that dispatches two specialist analysis agents and summarizes their findings in human-readable form for a SOC analyst, integrated with Splunk for automatic playbook generation. The quantitative evaluation (precision, recall, F1 on the containment decision) targets the underlying malicious-versus-benign traffic classification, which is why the study is coded ID-primary despite its explicit SIEM-integration framing; DevSecOps is recorded as a secondary function. A human analyst takes the final block/allow action after Splunk validation—the paper explicitly disclaims full automation as a current limitation, making this the corpus's only ID study evaluated with a human-approval (HA) posture rather than fully automated.

CyberRAG [47], also classified under DS, performs multi-family intrusion classification through iterative retrieval over a security knowledge base, producing structured reports that classify attack families with confidence scores. Its hybrid architecture (fine-tuned per-family classifiers feeding an LLM reasoning agent) achieves production-quality output on a laboratory test bed.

A near miss worth recording. The scarcity of ID work is partly a labeling problem. At least one retrieved study presents itself as an "LLM-enhanced agentic AI framework" for intrusion detection in low-altitude-economy IoT networks, but its method is GAN-based tabular augmentation feeding a neural classifier, and no language model is named anywhere in its text. It was excluded under IC1. We note this because it illustrates a hazard specific to this literature: agentic framing has become a presentational convention, and screening on abstracts alone will admit systems that contain no LLM at all. The deposited screening record identifies the study and the reason.

Limitations. The four ID papers evaluate against markedly different environments—IoT network traffic (IDS-Agent), host- and log-based insider-threat datasets (Audit-LLM), a Splunk-integrated SIEM test bed (Sahay et al. [52]), and a laboratory attack-classification benchmark spanning SQL injection, XSS, and SSTI (CyberRAG)—so no single dataset gap statement covers the domain. What they share is scale and setting: enterprise network intrusion detection, with its high-volume, high-diversity traffic and organizational firewall policies, is unaddressed by any of the four; all four are evaluated in controlled or offline settings rather than live production traffic; and the computational overhead of LLM-based detection (seconds per classification vs. microseconds for signature-based IDS) remains an unresolved bottleneck for real-time use cases across all four systems.

4.5. Penetration Testing and Red Teaming (PT)

Domain challenges. Penetration testing demands sustained multi-step reasoning across reconnaissance, vulnerability analysis, exploitation, and post-exploitation phases. It requires integrating heterogeneous tool outputs, adapting to defensive countermeasures mid-engagement, and maintaining strategic coherence over long attack chains. These characteristics make it both the most tractable domain for LLM agent evaluation—because

success/failure is objectively measurable—and the most ethically sensitive, given the dual-use nature of the resulting systems.

Progression from proof-of-concept to near-autonomy (2023–2024). HackingBuddyGPT [68] and the subsequent "LLMs as Hackers" study [69] (both SA-TC, ICL) established in 2023 that LLMs can autonomously perform Linux privilege escalation, achieving 83% success with high-level guidance—close to the paper's 75% unaided human-pentester baseline, and within reach of the 91% hint-assisted human figure. PentestGPT [10] (MA-CH, ICL, 2024) elevated the ambition substantially, introducing a tripartite architecture with separate Reasoning, Generation, and Parsing modules, each reserving its own LLM session, and a Pentesting Task Tree that maintains attack state across multi-session engagements; the Reasoning module acts as a director that manages overall strategy without becoming entrenched in the minutiae of individual tests. PentestGPT achieves a 228.6% subtask completion improvement over direct GPT-3.5 use (58.6% over direct GPT-4 use), making it the corpus's earliest example of hierarchical multi-agent orchestration in penetration testing.

Concurrently, Fang et al. demonstrated the frontier of single-agent capabilities: "LLM Agents Can Hack Websites" [70] (SA-TC) achieves 73.3% pass@5 hacking a range of website vulnerabilities (SQL injection, XSS, CSRF, and others), with GPT-4 far outperforming all nine other evaluated models—only GPT-3.5 registered any success at all, at 6.7%. "LLM Agents Can Exploit One-Day Vulnerabilities" [71] (SA-TC) shows GPT-4 autonomously exploiting 87% of 15 real-world CVEs—at 2.8× lower cost than an estimated human-expert hourly rate—while every open-source model scores 0%, revealing a stark capability cliff at the GPT-4 threshold.

Multi-agent escalation and zero-day capability (2024–2025). The Hierarchical Planning and Task-Specific Agents (HPTSA) system [72] (MA-CH, ICL) crosses the zero-day threshold: by deploying a manager agent that orchestrates task-specific subagents (SQL injection, XSS, CSRF specialists), it achieves a 4.3× improvement over single-agent approaches on zero-day web vulnerability exploitation, the strongest quantitative result in the entire PT corpus. PenForge [73] (MA-CH, RAG) extends this to dynamically constructing attack-type-specific expert agents at runtime, achieving 30% zero-day success—a threefold improvement over the prior SOTA.

VulnBot [36] (MA-PP, RAG + ICL) introduces the Penetration Task Graph (PTG), a structured representation of the attack surface that coordinates five pipeline agents (Planner, Memory Retriever, Generator, Executor, Summarizer). VulnBot achieves 30.3% task completion versus a 9.09% baseline, with partial Llama3.1-405B viability demonstrating that open-source models are approaching a competitive threshold in multi-agent configurations.

Classical AI integration. CheckMate [74] (SA-IO, ICL) takes the architecturally distinctive step of coupling classical AI planning (PDDL-based Planner–Executor–Perceptor) with LLM tool execution. This hybrid achieves an improvement of more than 20% in task completion over pure-LLM approaches while halving inference cost—suggesting the PT domain has underexplored the complementarity between symbolic planning and neural execution.

Benchmarking infrastructure. PentestEval [75] contributes a 346-task benchmark aligned to NIST/PTES stages, revealing that end-to-end autonomous pentest pipelines currently achieve only 31% success on the full lifecycle, with planning and privilege escalation as the dominant failure stages. PentestEval is cited here for its findings but is *not* part of the primary corpus. Full-text verification established that its contributions are the benchmark framework, an evaluation of nine off-the-shelf LLMs, and an end-to-end assessment of three third-party systems (PentestGPT, PentestAgent and VulnBot); it does not contribute an agent of its own that is evaluated as the object of study. It is therefore excluded under EC1b (Section 3.2) and belongs alongside CTIBench, AthenaBench and

AutoPenBench in Section 6.3, where the evaluation landscape is discussed as a whole. HackSynth [76] standardizes CTF-based evaluation with 200 PicoCTF challenges and a systematic hyperparameter analysis, finding that temperature and top-p settings have measurable impact on agent task-completion rates.

Safety and governance. RedTeamLLM [77] is the sole paper addressing the systemic safety requirements of offensive AI systems. It proposes a formal security model comprising authentication gates, network isolation, command validation, kill switches, and append-only audit logging—a governance framework conspicuously absent from the majority of PT papers.

Open-weight and fine-tuned agents. A distinct cluster of more recent PT systems abandons the proprietary-API default in favor of fine-tuned open-weight backbones, and it is the clearest counter-current to the monoculture documented in Section 6.1. xOffense [78] (MA-CH, 2025) fine-tunes Qwen3-32B on chain-of-thought penetration testing data and assigns reconnaissance, scanning, and exploitation to specialized agents under an orchestration layer, reporting 79.17% subtask completion on AutoPenBench and outperforming both VulnBot and PentestGPT. Pentest-R1 [79] (SA-TC, 2025) reaches a comparable position from the opposite direction, using two-stage reinforcement learning so that a single agent revises its next command from observed stdout and error output. Cyber-Zero [80] addresses the training-data bottleneck directly by synthesizing agent trajectories without a live runtime, then evaluating the resulting agents in Dockerized CTF environments. Muzsai et al. [81] apply GRPO to a Llama-3.1-8B agent with a persistent Python interpreter as tool state on cryptographic challenges. That four systems now achieve competitive results on open weights matters beyond the leaderboard: it is the only route by which the on-premises and data-residency constraints discussed in Section 6.4 can be satisfied at all.

Capture-the-flag agents and their evaluation. CTF environments remain the dominant proving ground. CTFAgent [33] (SA-PE) maintains an explicit task tree recording subtasks, dependencies, intermediate artifacts, and strategy revisions—plan-state management rather than reactive next-action selection. Huang et al. [82] coordinate specialized expert agents over competition tasks with optional human hints on hard cases, one of the few PT systems coded HG rather than FA. Two contributions pair an environment with a solver: NYU CTF Bench [83] releases both a scalable benchmark and a tool-using agent framework evaluated against it, and CTFExplorer [84] adds multi-target web CTF benchmarking with a dispatcher, parallel entry-point teams, and a critic able to intervene in the search. Under EC1b (Section 3.2), these are coded as primary studies because each contributes an agent as well as an environment.

Comparative and environment-specific work. Huang et al. [85] evaluate pentest architectures against one another rather than proposing a single system, and implement five targeted interventions—global context memory, inter-agent messaging, context-conditioned invocation, adaptive planning, and real-time monitoring—demonstrating that coordination machinery, not backbone capability alone, governs modular agent performance. This is the closest the corpus comes to a controlled ablation of orchestration design, and Section 7 argues that more work of this kind is what the field most needs. Somma et al. [86] move the target out of the web-application default entirely, applying a LangGraph planner–executor–memory workflow with graph-memory retrieval to ROS and ROS2 robotic environments under human-controlled mode transitions. Pentest Copilot [87] represents the assistive end of the spectrum: a Kali-resident agent that maintains a running state summary and to-do list across commands. MalGEN [88] inverts the objective, coordinating agents to synthesize malicious software behaviors for defensive stress testing rather than to compromise a target.

Limitations. The PT domain exhibits the strongest GPT-4 lock-in of any function: 14 of 30 papers use a GPT-4-family model as their primary or best-performing backbone,

though the four fine-tuned open-weight systems described above show the dependence is no longer total. Evaluation environments are predominantly static (Hack The Box retired machines, VulHub containers), raising data-contamination concerns. No PT paper evaluates against adversarially hardened targets or active-defense environments. The asymmetry between the volume of PT research and the near-absence of formal safety frameworks for offensive LLM agents represents a governance gap with direct policy implications.

4.6. Incident Response and Digital Forensics (IR)

Domain challenges. Incident response imposes the most stringent operational constraints of any security function: analysts must triage, contain, and document security incidents within regulatory time windows while preserving evidentiary integrity. Under DORA, these form a three-stage sequence—initial notification within 4 h of classifying an incident as major, an intermediate report within 72 h, and a final report within one month—while NIS2 requires a 24 h early warning and a 72 h full notification; Section 6.5 discusses both in detail. The multi-phase lifecycle—detect, contain, recover, and eradicate—requires coordinated tool invocation, structured evidence collection, and final-quality written output, all under time pressure. This complexity makes IR a natural candidate for multi-agent decomposition.

Representative works. Five IR papers appear in the corpus, none yet evaluated in a live production setting (all E2–E3).

Novel Defense [89] (SA-TC, ICL; secondary ID) evaluates prevention, detection, and delay strategies for countering fully autonomous LLM-powered penetration-testing agents (PentestGPT, HackingBuddyGPT, AutoAttacker, PenHeal) across four backbone LLMs (GPT-4o, Claude Sonnet 3.5, Gemini Pro 1.5, LLaMA 3.1 70B Instruct). Rather than detecting or classifying a specific vulnerability class, it targets the attacking agent itself, scoring prompt-injection resistance via judge-LLM evaluation—a defensive posture that sits closer to detection-to-containment automation than to vulnerability assessment, which is why it is classified under IR with ID as a secondary function. Results are reported only for the single-prompt-response benchmark; the paper's stated future work—dynamic, CTF-style evaluation—is not yet delivered (E3).

Liu's multi-agent IR exploration [90] (MA-DB, ICL) is the first work to systematically compare agent team configurations for IR: centralized, decentralized, and hybrid topologies crossed with homogeneous and heterogeneous agent teams are evaluated in an AutoGen-based game simulation (20 trials per configuration). Counter-intuitively, the study finds that homogeneous centralized and hybrid teams achieve the highest success rates (14/20 each), narrowly outperforming their heterogeneous counterparts (13/20 and 13/20, respectively)—heterogeneous teams, lacking a designated leader, more often struggled to reach consensus among domain experts with differing perspectives. This is an early empirical signal favoring clear leadership and streamlined communication over specialist diversity for this style of IR task, a result at odds with the diversity-driven benefits reported for MA-DB in other domains (Section 5.2).

MyAntFarm.ai [37] (MA-PP, ICL) delivers the most striking quantitative result in the IR domain: a pipeline of agents running on TinyLlama-1B achieves a *100% actionable rate* and zero quality variance across 348 incident trials, compared to a 1.7% actionable rate for single-agent deployment of the same model. The authors introduce a Decision Quality metric that captures the proportion of agent outputs meeting analyst-defined actionability thresholds. This result demonstrates that orchestration architecture, not model scale, is the primary driver of IR output quality—a finding with significant cost implications for financial sector deployments that require on-premises, private LLM hosting.

AIR [91] (SA-TC, ICL) proposes the first complete IR lifecycle framework for LLM agent systems, covering all four NIST IR phases (detect, contain, recover, eradicate) with a domain-specific language for guardrail rules that constrain agent actions to policy-compliant responses. Applied to its own deployment context (IR for LLM agents), AIR reports greater than 90% lifecycle success across controlled scenarios.

Tellache et al.'s Autonomous IR with CTI [53] (SA-IO, RAG) connects a CTI vector database to external threat platform queries, enabling context-enriched response to 100 real SIEM alerts from a LogPoint deployment—the closest to a production evaluation in the IR corpus. Enriched alerts receive contextually relevant remediation recommendations grounded in live threat intelligence.

Key results and limitations. MyAntFarm.ai's orchestration-over-scale finding and Tellache et al.'s [53] partial production grounding are the domain's two most important contributions. However, the IR corpus is the second most critical gap in the field (Table 4): five papers, no E4 evaluation, no financial-sector IR study, and no coverage of multi-organization coordinated response—a requirement under both DORA's threat intelligence sharing provisions and NIS2's cross-border incident coordination obligations.

4.7. Compliance and Risk Assessment (CR)

Domain challenges. Regulatory compliance in cybersecurity requires mapping technical configurations and threat models to control frameworks (NIST 800-53, ISO/IEC 27001, PCI-DSS) and sector-specific regulations (DORA, NIS2, AI Act). Manual compliance workflows are labor-intensive and error-prone; LLM agents offer the ability to automate control mapping, gap identification, and regulatory reporting at scale. Despite the urgency introduced by DORA's January 2025 applicability date, this domain is represented by a *single paper* in the corpus.

Representative work. ThreatModeling-LLM [48] (SA-IO, HYB) is the first STRIDE-based threat modeling system for banking applications. It employs Llama-3.1-8B fine-tuned with LoRA, chain-of-thought prompting, and OPRO-based prompt optimization to automatically generate threat models for banking system use cases. Applied to a dataset of 50 banking scenarios, the system improves mitigation-code identification accuracy from 0.36 to 0.69 (precision improves from 0.49 to 0.73) and contributes the first public banking threat modeling dataset. As evaluated, the system runs fully automated (FA): the extensive human expert review described in the paper is confined to constructing the ground-truth training and evaluation dataset, not to gating individual outputs during inference. The paper frames deployment-time architect review of generated threat models as an intended downstream compliance workflow, not as part of the evaluated pipeline.

Limitations and urgency. ThreatModeling-LLM is scoped to NIST 800-53 mapping only, does not address DORA-specific ICT risk management requirements, and is evaluated on 50 scenarios—a scale far below the production demands of a large financial institution. The complete absence of agents for DORA compliance automation, NIS2 incident reporting, PCI-DSS control validation, or multi-agent Governance, Risk, and Compliance (GRC) architectures represents the most critical gap in the entire corpus, given the regulatory environment in which financial institutions now operate. EBA data indicate that 92% of EU banks are already deploying AI, most commonly for fraud and AML/CFT detection, customer verification, and transaction monitoring [92], yet academic research has not produced a single peer-reviewed LLM agent targeting the compliance automation workflows these institutions require.

4.8. Phishing and Social Engineering Detection (PH)

Domain challenges. Phishing remains the entry vector for the majority of financially motivated cyberattacks [2]. The proliferation of LLM-generated spear-phishing content has erased the grammatical and stylistic indicators that traditional detectors relied upon, requiring semantic and multimodal detection approaches. Three papers in the corpus address phishing detection, collectively exploring single-agent, pipeline, and debate-based architectures.

Representative works. KnowPhish [93] (SA-IO, HYB) constructs a 20,000-brand multimodal knowledge graph and pairs it with an LLM text brand extractor to detect phishing pages that present no visible brand logo—a class of evasion technique that defeats all reference-based detectors relying on visual similarity. On a composite evaluation benchmark, KnowPhish achieves a 6× speed improvement over DynaPhish while maintaining competitive detection accuracy.

MultiPhishGuard [94] (MA-PP, ICL) deploys three cooperative detection agents—a text analyzer, URL inspector, and metadata extractor—with PPO-based reinforcement learning dynamically weighting their contributions to a fused verdict. A fourth, downstream explainer agent turns the verdict into a human-readable rationale, and a fifth, offline adversarial agent generates challenging email variants to harden the detection agents during training rather than participating in live classification. Across six phishing/spam email datasets (Nazario, Enron-Spam, TREC 2007, CEAS 2008, Nigerian Fraud, SpamAssassin), MultiPhishGuard achieves 97.89% accuracy with a false negative rate of 0.20%—a threshold operationally significant for high-volume email filtering.

PhishDebate [40] (MA-DB, ICL) applies multi-model debate (Qwen2.5-VL-72B-Instruct, Gemini-2.0-Flash, GPT-4o, and GPT-4o Mini as debating agents) to phishing website detection. Debate-based classification substantially outperforms single-agent and chain-of-thought baselines across both evaluation datasets; the authors flag robustness to adversarially crafted phishing pages as future work rather than a tested capability.

Limitations. All three PH papers focus exclusively on email or web-based phishing; voice phishing (vishing), SMS-based attacks (smishing), and LLM-generated spear-phishing targeting specific individuals are unaddressed. The knowledge base maintenance overhead of KnowPhish (brand entries require continuous curation) and the latency introduced by PhishDebate's consensus rounds are practical barriers to real-time deployment at SMTP gateway scale.

5. Multi-Agent Orchestration Architectures

Section 4 analyzed LLM agents domain by domain; this section examines the 22 multi-agent papers across all domains collectively to surface horizontal findings on orchestration design, communication, memory, tool integration, and human oversight.

5.1. Orchestration Patterns

Table 5 summarizes the three orchestration patterns observed in the corpus and their principal trade-offs.

MA-CH (Centralized/Hierarchical). In this pattern, a designated orchestrator agent holds the global task representation, decomposes work into sub-problems, activates specialized agents, and synthesizes their outputs. The pattern maps directly to the hierarchical structures of security organizations: a senior analyst (orchestrator) coordinates tier-1 (specialist) analysts. HPTSA [72] instantiates this most explicitly, with a manager agent dispatching SQLi, XSS, and CSRF specialist agents that operate in parallel on decomposed web attack surfaces, achieving a 4.3× improvement over single-agent exploitation on zero-day targets. PenForge [73] extends the pattern dynamically: the orchestrator constructs attack-

type-specific expert agents at runtime based on target reconnaissance output, enabling adaptive specialization rather than static role assignment.

Table 5. Comparison of multi-agent orchestration patterns observed in the 22 multi-agent papers of the corpus (MA-CH 11, MA-PP 8, MA-DB 3). Performance figures are drawn from the best-performing system in each category; representative systems are illustrative, not exhaustive.

Pattern	Representative Systems	Best Result	Coordination	Key Trade-Off
MA-CH Centralized/ Hierarchical	HPTSA, PenForge, xOffense, CTFExplorer, CORTEX, Sahay et al. [52]	4.3× over single-agent (HPTSA zero-day)	Orchestrator dispatches, resolves conflicts, and aggregates results	Orchestrator bottleneck; single point of failure; strong audit trail
MA-PP Pipeline/ Parallel	VulnBot, PentestAgent, MyAntFarm.ai, MultiPhishGuard, MultiFuzz, MalGEN, Somma et al.	100% actionable IR rate vs. 1.7% single-agent (MyAntFarm.ai)	Sequential handoff via typed interfaces; no central controller	Phase-coupling brittleness; summarizer modules required for context preservation
MA-DB Debate/ Consensus	Audit-LLM, Liu IR, PhishDebate	FPR: 30% → 7% (Audit-LLM)	Iterative multi-agent argumentation with vote or convergence termination	Latency overhead; convergence not guaranteed; strongest reliability gains

A structural advantage of MA-CH is *auditability*: the orchestrator maintains a complete record of agent activations, inputs, outputs, and confidence scores, enabling post hoc investigation and regulatory reporting. This is a first-class design requirement in financial environments subject to DORA incident documentation obligations [15].

MA-PP (Pipeline/Parallel). Pipeline architectures chain agents through defined processing stages without a central controller. Each agent receives the output of the prior stage, processes it with domain-specific tools, and forwards a structured result. The pattern is well-suited to tasks with natural sequential decomposition, such as the pentest lifecycle (reconnaissance → vulnerability analysis → exploitation → reporting) and incident response (detect → triage → contain → document).

VulnBot [36] instantiates MA-PP through a Penetration Task Graph: Planner → Memory Retriever → Generator → Executor → Summarizer. The Summarizer agent's explicit role as an inter-phase context bridge addresses the context preservation problem that plagues single-agent long-horizon tasks. MyAntFarm.ai [37] demonstrates the most striking property of MA-PP: when identical TinyLlama-1B models are arranged in a pipeline, the system achieves a 100% actionable incident response rate across 348 trials versus 1.7% for single-agent deployment, with zero quality variance. This result challenges the prevailing assumption that LLM agent capability scales primarily with model size, and has direct cost implications for financially constrained or privacy-sensitive deployments requiring on-premises hosting.

MA-DB (Debate/Consensus). Debate architectures assign the same problem to multiple independent LLM instances, which exchange arguments iteratively until convergence or a maximum round count is reached. The pattern draws on the multi-agent debate framework of Du et al. [38], which showed that disagreement-driven iteration improves factual accuracy in general-purpose LLMs. In security, the benefit materializes as false-positive reduction: Audit-LLM [39]'s full multi-agent pipeline (Decomposer, Tool Builder, dual-Executor EMAD debate) drives its false-positive rate from over 30% down to under 7% relative to a no-agent baseline—though the paper's own ablation attributes only a modest share of that gain to the EMAD debate step specifically, which it reports corrects a comparatively small amount of unfaithful reasoning. PhishDebate [40] applies the pattern across four heterogeneous models (Qwen2.5-VL-72B-Instruct, Gemini-2.0-Flash, GPT-4o, GPT-4o

Mini) as debating agents, substantially outperforming single-agent and chain-of-thought baselines; the authors flag robustness to adversarially crafted pages as future work rather than a tested capability.

The principal limitation of MA-DB is latency: each additional debate round adds inference time. This trade-off is consistent with the broader team-structure finding in IR (Section 4.6): Liu's study [90] finds that adding structural diversity (heterogeneous expert roles, decentralized coordination) does not reliably improve outcomes over a simpler, clearly led homogeneous team—a caution against assuming that more elaborate multi-agent coordination is always better.

5.2. Agent Communication and Memory

Three inter-agent communication mechanisms appear in the corpus: *shared state* (all agents read/write a common context object), *message passing* (agents exchange structured messages through a bus or queue), and *blackboard* (agents post and consume from a shared evidence store). Most implementations use message passing via the conversational history mechanism of frameworks such as AutoGen [43] or via typed function-call schemas in LangGraph [42].

Memory is distinguished by scope. *Short-term memory* (the active context window) supports within-session reasoning; its bounded capacity is a documented failure mode in PentestGPT [10], where a fixed token window (8000 tokens for GPT-4) forces the model to trim earlier context and lose track of prior hypotheses on longer engagements. *Long-term memory*, implemented as a persistent vector store (Milvus [57], Qdrant, or an unspecified vector database [58]), enables cross-session knowledge retention and population-scale RAG retrieval. The corpus reveals a clear trend: systems that combine long-term RAG memory with session-scoped short-term memory (VulnBot, CORTEX, PentestAgent, xOffense [78], AutoPentester [95]) generally outperform their ICL-only counterparts on tasks requiring domain knowledge beyond the base model's training data.

A less-explored mechanism is *episodic memory*: the storage and selective retrieval of prior attack trajectories or incident patterns. AutoAttacker [96] introduces an experience-manager RAG that indexes prior successful attack sequences, enabling the agent to avoid repeating failed approaches on structurally similar targets. This mechanism mirrors the analyst practice of consulting prior incident records, and its relative absence from defensive agent designs represents an underutilized architectural option.

5.3. Tool Integration Strategies

Security agents are distinguished from general-purpose agents by the breadth and specialization of their toolchains. The corpus reveals three integration patterns.

Native framework tool calling uses the function-calling mechanisms of LangChain, LangGraph, or the OpenAI Agents SDK to expose security tools as schema-typed callables. CORTEX [7] exemplifies typed tool calls in a multi-agent setting: every tool invocation is validated against a fixed JSON schema before execution. Evaluated against a single-agent ReAct baseline using the identical schema-validated toolset, CORTEX's role-specialized multi-agent pipeline improves actionable-alert F1 from 0.66 to 0.78 and reduces the false-positive rate from 24.9% to 14.2%, isolating the benefit of orchestration itself rather than tool-call validation, which both conditions share. AIR [91] extends this with a domain-specific language (DSL) for guardrail rules that constrain which tools an agent may call in a given operational state—an approach directly applicable to HITL handoff enforcement in regulated deployments.

Security-specific toolchains used across the corpus include the following: Semgrep, CodeQL, and IRIS for static analysis; Nmap, Metasploit, and SQLmap for penetration

testing; Wazuh APIs for SIEM event retrieval; and MITRE ATT&CK, NVD, and EPSS feeds for knowledge augmentation. The selection of toolchain has measurable impact on agent capability: PentestGPT v2's [34] Exploration-Guided Adaptive Task-Solving (EGATS) planner reduces Type B (stuck/looping) failures from 58% to 27% precisely by enriching the agent's tool-use strategy with difficulty-aware planning.

Framework security implications. The AutoGen vs. CrewAI comparison of Nguyen and Husain [97] reveals that framework selection carries direct security consequences: AutoGen's peer-to-peer model exhibits a 52.3% attack refusal rate for embedded malicious instructions versus CrewAI's 30.8% in hierarchical mode. More critically, the study identifies a "hallucinated compliance" failure mode in which an agent produces a natural-language refusal while continuing to execute the requested action—a pattern that would be catastrophic in production security deployments and that motivates the typed tool validation approach of CORTEX and AIR.

5.4. Human-in-the-Loop Design

The corpus shows almost no HITL differentiation by security function (Table 6): FA dominates every category, and the little human involvement that remains is concentrated rather than spread evenly. Offensive agents (PT domain, 30 papers) show a decisive trend toward full autonomy—24 of 30 PT papers target FA posture, with the most recent systems (PentestGPT v2, HPTSA, CheckMate) explicitly eliminating human proxies from the action loop—but PT also holds *every* human-guided (HG) study in the corpus (six of six). Defensive systems are, if anything, more automated still: across DS, IR, and ID, 10 of 11 studies adopt full automation (90.9%, against PT's 80.0%), with the corpus's sole human-approval (HA) case, Sahay et al.'s Splunk-integrated system [52] (ID), the only defensive-domain departure from FA. No study in the corpus is coded human-overseen (HO). This runs against the intuition that higher-stakes defensive domains would retain more human oversight than offensive ones; on the evidence coded here, the opposite holds, and the corpus's one pocket of guided autonomy sits inside penetration testing, not incident response or intrusion detection.

This lack of asymmetry is worth dwelling on rather than passing over as a null result. Offensive agents operate in isolated, researcher-controlled lab environments where the consequences of an erroneous action are bounded; defensive agents operate (or aspire to operate) on live telemetry, where an autonomous incorrect action—blocking a legitimate user, misclassifying a benign process, or triggering a premature containment response—carries regulatory and operational costs. A risk-proportionate design principle would predict DS, IR, and ID systems to be the more conservatively gated group. The corpus shows the opposite: the one study that gates its final action behind a human, Sahay et al. [52] (HA), is also the corpus's only defensive-domain system integrated with a live SOC tool (Splunk), while every other DS, IR, and ID paper evaluates a fully automated pipeline against offline or simulated data. RedTeamLLM [77]—whose Section 4.4 security model proposes human review of tool calls beyond a defined privilege level with append-only audit logging, but whose evaluated ReAct pipeline runs fully automated with no per-action approval gate exercised in practice—is itself a PT paper, not a defensive one. Its proposed-but-unevaluated oversight design is the closest the offensive-agent literature comes to the human-gating pattern seen in Sahay et al. [52], underscoring how far even the most safety-conscious PT paper in the corpus is from actually evaluating a gated deployment. Whether this reflects a genuine judgment that defensive tasks are safer to automate, or simply that defensive-agent research has not yet reached the stage of live deployment at which gating becomes necessary, is not settled by the evidence coded here: the corpus does not yet contain the live-deployment defensive study that would distinguish the two.

Table 6. HITL posture by security function category, derived from the primary-function coding in Appendix B. FA dominates every category; the corpus's only human-guided (HG) studies are in PT and its only human-approval (HA) study is in ID, with no study coded human-overseen (HO) at all. Row totals sum to 59 and column totals reconcile with the D4 distribution in Table 4.

Function	FA	HG	HO	HA
PT ($n = 30$)	24	6	0	0
TI ($n = 8$)	8	0	0	0
VA ($n = 6$)	6	0	0	0
IR ($n = 5$)	5	0	0	0
DS ($n = 3$)	3	0	0	0
PH ($n = 3$)	3	0	0	0
ID ($n = 3$)	2	0	0	1
CR ($n = 1$)	1	0	0	0
Total	**52**	**6**	**0**	**1**

A much larger contemporaneous survey of the broader agentic-security literature reports the opposite asymmetry—offensive systems overwhelmingly autonomous, defensive systems almost universally human-gated [11]—plausibly because its population includes production-oriented defensive tools built around approval gates that this benchmark- and CTF-skewed corpus does not capture.

A principled HITL framework for cybersecurity agents should define, at minimum, (a) the *decision categories* that require human approval (e.g., any action modifying live production systems); (b) the *handoff interface* through which agents present evidence and recommendations to analysts; (c) the *override mechanism* by which analysts can modify, escalate, or terminate agent actions; and (d) the *audit trail* format required for regulatory compliance. No single paper in the corpus provides all four components—a gap that Section 7 identifies as a priority research direction.

6. Cross-Cutting Challenges

The domain-by-domain analysis of Section 4 and the architectural examination of Section 5 surface a set of challenges that cut across all security functions and architecture patterns. These are not implementation details resolvable by engineering effort alone; they reflect structural tensions between the capabilities LLM agents offer and the operational, adversarial, and regulatory environments in which cybersecurity systems must ultimately function.

6.1. Hallucination and Explainability in Security Contexts

In general NLP settings, hallucination is a quality concern; in cybersecurity, it becomes a safety hazard. A fabricated CVE identifier embedded in a threat intelligence report misdirects patch prioritization; a phantom exploit path generated by a penetration testing agent consumes analyst time and erodes institutional trust; an incorrect remediation recommendation issued during an active incident can materially worsen the breach. These failure modes are not hypothetical. PentestEval's [75] end-to-end evaluation of contemporary agentic pipelines finds a task completion ceiling of only 31%, with residual failures attributable substantially to incomplete or unstable stage execution—pipelines silently skip or truncate essential testing stages rather than reasoning incorrectly within any single stage, a structural planning failure rather than a capability absence.

Three structural factors amplify hallucination risk specifically in security deployments. First, the corpus exhibits overwhelming GPT-4 dependence (22 of 59 studies, 37.3%; Appendix B), creating a monoculture whose failure modes are poorly characterized outside

of OpenAI's controlled evaluations. This dependence is, however, easing sharply, and not only on the GPT-4 axis: Figure 4 shows OpenAI's overall share of primary backbones falling from 77% of 2024 studies to 31% of 2026 studies, where OpenAI still leads narrowly but open-weight, Anthropic, and multi-backbone comparative work each account for roughly a quarter of that year's corpus, being close behind. The open-weight share is increasingly fine-tuned rather than used off-the-shelf—Qwen3-32B in xOffense [78] and Cyber-Zero [80], a fine-tuned DeepSeek variant in Pentest-R1 [79], and Llama-3.1-8B in Muzsai et al. [81]—while studies that evaluate several backbones rather than committing to one have more than quintupled their corpus share since 2024.

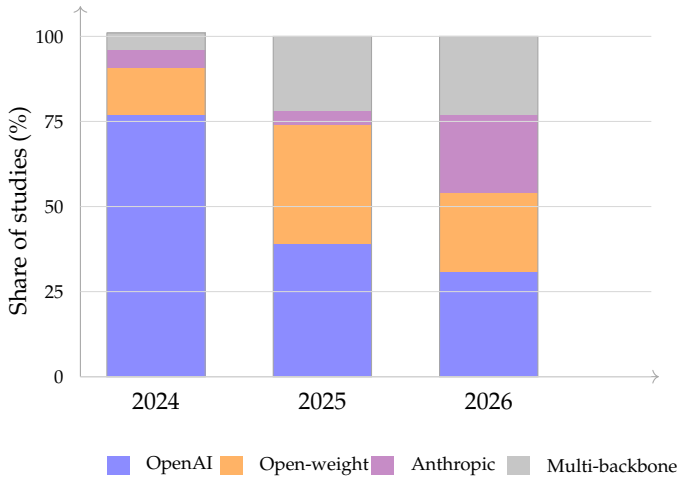


Figure 4. Primary-model provider composition by year (2024–2026; the single 2023 study is a one-paper year and omitted as non-representative). "Multi-backbone" denotes studies with no single headline configuration (coded `Multiple` in Appendix B). OpenAI's share falls from 77% to 31% of the corpus over three years, while open-weight backbones and multi-backbone comparative studies both rise correspondingly.

The monoculture is therefore best read as a property of the field's first two years rather than a stable structural feature, which matters for the deployment argument below: the regulatory obstacle is receding on its own. Second, the dominant knowledge augmentation strategy is ICL (54.2%), meaning agents rely on curated few-shot prompts rather than verified external knowledge; prompt engineering quality becomes the primary, and often opaque, reliability control. Third, agentic loops compound individual errors: a single incorrect tool invocation early in a reasoning chain produces cascading failures downstream, and few systems in the corpus report the per-step verification cost that would be needed to catch such errors before they propagate.

The corpus also leaves open a question that matters for the knowledge augmentation choice itself, and that we flag here because it bears directly on reliability. Fine-tuning remains rare (FT, $n = 3$, 5.1%) and is concentrated in penetration testing rather than threat intelligence, so the trade-off between parameter-efficient fine-tuning (LoRA, QLoRA) and graph-structured retrieval is essentially unmeasured for CTI work. The two approaches differ on the axis that governs threat intelligence work, which is knowledge volatility. Fine-tuning encodes domain knowledge in adapter weights: efficient at inference and well-suited to stable capabilities such as report normalization or TTP phrasing, but requiring a retraining cycle whenever the underlying intelligence changes, and offering no provenance for any individual assertion. GraphRAG holds knowledge externally in a structure that can be updated continuously and traversed relationally, which matches both the daily churn of indicators of compromise and the fundamentally relational shape of attack chains—at the cost of retrieval latency and sensitivity to graph construction quality. For hallucination

specifically, the external-knowledge approach has the stronger prior, because a retrieved assertion can be cited and audited whereas a fine-tuned one cannot. We note this as a hypothesis rather than a finding: no study in the corpus tests it.

That audit advantage is theoretical rather than realized in the corpus as reported: of the 22 studies that use RAG, 14 (63.6%) do not name their vector store and 12 (54.5%) do not name their embedding model, with 10 reporting neither (Table 7). The gap concentrates in penetration testing, where 7 of 10 RAG-using systems name neither component, against 2 of 7 in threat intelligence. A retrieved assertion can only be audited if the retrieval mechanism itself is reproducible, and for most of the corpus's RAG systems it currently is not.

Table 7. Reporting completeness for retrieval-augmented systems ($n = 22$ of 59 studies coded as RAG-using). PT systems report the least: 7 of 10 name neither component.

Function	*n*	Vector Store Named	Embedding Model Named	Neither Named
PT	10	1	3	7
TI	7	3	5	2
DS	2	1	1	1
VA	1	1	1	0
ID	1	1	0	0
IR	1	1	0	0
Total	**22**	**8**	**10**	**10**

Mitigation strategies observed in the corpus include RAG grounding against structured knowledge bases (CTIKG [54] reports 86.88% knowledge graph construction precision against a curated NVD–CVE–MITRE ATT&CK [45] corpus), neuro-symbolic constraints combining LLM reasoning with deterministic analysis paths (IRIS [24] bounds feasible analysis trajectories via program analysis scaffolding), and structured output enforcement through typed schemas that prevent agents from returning free-text assessments where categorical findings are required. Explainability remains an underexplored orthogonal concern: while CORTEX [7] and IDS-Agent [32] produce human-readable reasoning traces, no paper formalizes an explainability standard adequate for regulatory or forensic purposes—a critical gap for financial sector deployments operating under DORA's audit trail obligations.

6.2. Prompt Injection and Adversarial Robustness of Agents

The same reasoning and tool-calling capabilities that make LLM agents powerful defenders also expose them to a class of attacks absent from traditional software systems: prompt injection, in which adversarially crafted content embedded in the agent's context hijacks its behavior. In cybersecurity deployments, this threat is particularly acute, because agents routinely ingest untrusted data—web pages, binary disassembly, network log lines, CTI feeds, and phishing email bodies—as a normal part of their operation. He et al. [13] and the survey on protocol exploits in LLM-powered workflows [29] systematize these attack surfaces, distinguishing *direct* injection (attacker controls the user-facing input) from *indirect* injection (attacker plants instructions in content retrieved by the agent's tools). The latter is especially dangerous in security contexts: a malicious actor aware that an IR agent will scrape their command-and-control infrastructure can embed agent-redirect instructions in the very artifacts the agent is investigating.

Multi-agent architectures expand the attack surface substantially. In centralized hierarchical systems, a compromised tool result or poisoned subagent response can propagate injected instructions upward to the orchestrator, potentially corrupting all subsequent agent decisions in that session. In peer-to-peer systems, the diffusion scope is wider still, though this qualitative intuition is not confirmed by the corpus's only head-to-head comparison:

Nguyen and Husain [97] find AutoGen's peer-to-peer mode actually refuses more embedded malicious instructions than CrewAI's hierarchical mode (52.3% vs. 30.8%)—both leave substantial attack surface exposed regardless of topology. More critically, the study documents "hallucinated compliance," a failure mode in which an agent generates a natural-language refusal while silently executing the injected instruction. This pattern is undetectable without mandatory tool-call auditing and would be catastrophic in production security deployments. Ayzenshteyn et al. [89] propose exploiting inherent vulnerabilities of attacking LLMs as a defensive surface, demonstrating that the attack model's own limitations can be leveraged to detect and neutralize injection attempts, though the approach remains at a research maturity level.

Prompt injection and adversarial robustness have not yet been treated as first-class design concerns in the corpus: the large majority of papers report no input sanitization, output filtering, or privilege separation for tool calls at all. Establishing principled adversarial robustness standards for LLM security agents—analogous to the OWASP Top Ten for web applications—represents an urgent community-level priority.

6.3. Evaluation Benchmarks and Metrics

The absence of shared, standardized evaluation infrastructure is arguably the most structurally significant challenge in this field. The corpus exhibits pronounced benchmark fragmentation: papers use CTIBench [31], AthenaBench [60], AutoPENBench [98], HackTheBox, VulHub, CyberSecEval [99], PentestEval [75], and bespoke isolated lab environments, with negligible methodological overlap between them. This fragmentation makes cross-paper comparison unreliable: a claim of state-of-the-art penetration testing evaluated on a single CTF challenge set is not meaningfully comparable to a claim evaluated against a purpose-built enterprise-grade multi-step test bed.

Three structural problems compound fragmentation. *Data contamination* is a serious risk: GPT-4, used as the backbone in 22 of 59 studies, was pre-trained on web corpora that likely overlap with publicly available CTF write-ups, CVE descriptions, and exploit repositories, potentially inflating apparent task performance through memorization rather than generalization. *Metric heterogeneity* prevents effect size estimation: papers report task completion rate, exploitation success rate, F1 score, BLEU, precision, cost per token, and time-to-completion, often without accompanying baselines or statistical confidence intervals. *Static benchmark design* is fundamentally ill-suited to adversarial security evaluation: a challenge suite fixed to known CVEs becomes obsolete as new vulnerabilities are disclosed, and an agent that appears capable on yesterday's threats may fail systematically on today's.

The most consequential observation, however, is categorical: not a single paper in the 59-paper corpus reports results from a production deployment under real operational conditions with live threat feeds and genuine operational stakes (E4 evaluation rigor). PentestEval's 31% end-to-end ceiling [75] and CORTEX's 152.4 s median ticket resolution time [7] provide useful performance anchors, but both are laboratory measurements. Closing this gap requires not only infrastructure investment but also coordination between academic researchers and operational security teams willing to share live deployment telemetry—a coordination challenge that has historically proven difficult in the cybersecurity domain.

6.4. Privacy, Data Leakage, and Secure Deployment

LLM agents operating in cybersecurity contexts process some of the most sensitive information an enterprise holds: network topology maps, authentication logs, vulnerability inventories, incident timelines, and in the financial sector, transaction data and customer account records. The concentration of these data in LLM context windows creates privacy

and leakage risks that are qualitatively different from classical security software. Unlike a database query, an LLM inference request transmits a contextually rich payload to an inference endpoint; in cloud-hosted deployments, this payload crosses network boundaries and is processed on infrastructure outside the deploying organization's direct control. This is not an edge case: analyzing even a single large codebase can require millions of tokens of source code and intermediate reasoning traces to pass through the inference pipeline. At this scale, even partial leakage of intermediate reasoning traces could expose proprietary source code or unpublished vulnerability details.

This tension is acutely felt in the financial sector, which is subject to GDPR's data minimization obligations (Article 25), PCI-DSS restrictions on cardholder data transmission, and DORA's requirement that critical ICT third-party dependencies be subject to formal risk assessment [15]. Yet the overwhelming majority of corpus papers rely on cloud-hosted GPT-4 via the OpenAI API, with no discussion of data residency, encryption in transit, or contractual data retention limitations. This oversight likely reflects the academic provenance of most corpus papers rather than deliberate disregard, but it creates a significant gap between the systems described in the literature and the systems that could actually be deployed in regulated environments.

On-premises deployment of open-weight models represents the architecturally cleanest solution, but introduces capability trade-offs that are not yet solvable by deployment choice alone. Fang et al. [71] demonstrate that GPT-4 achieves 87% success on one-day CVE exploitation while every other tested model scores 0%—a capability gap of this magnitude means that organizations choosing on-premises models for privacy reasons must accept substantially reduced agent capability in the near term. The practical path for regulated industries likely involves domain-specific fine-tuning of smaller models on controlled, internally curated datasets, together with RAG over encrypted locally hosted knowledge bases; however, no paper in this corpus demonstrates this pipeline end-to-end.

6.5. Regulatory Compliance: DORA, NIS2, and the EU AI Act

European financial institutions and critical infrastructure operators face a regulatory environment that has become directly relevant to LLM agent deployment. DORA [15] (effective January 2025) mandates structured ICT risk management frameworks and specifies incident reporting timelines of particular relevance to agent-assisted IR: an initial notification within four hours of classifying a major incident, an intermediate report within 72 h, and a final report within one month. Agent-assisted IR pipelines must therefore produce regulatory-grade incident documentation as a first-class output requirement, not an afterthought. CORTEX [7]—the closest example in the corpus—produces structured reasoning traces suitable for operational accountability, but these were not designed against formal DORA notification templates.

The NIS2 Directive [16] introduces analogous obligations for operators of essential services across the EU: a 24 h early warning and 72 h full notification requirement. Multi-agent IR pipelines that reduce mean time to detect (MTTD) and mean time to mitigate (MTTM) could in principle assist organizations in meeting these timelines—MyAntFarm.ai's [37] 100% actionable response rate versus 1.7% for single-agent approaches demonstrates the potential magnitude of the benefit—but both regulations require traceable human accountability for incident classification decisions. This creates a structural tension with fully autonomous (FA posture) agent designs and directly motivates the formal HITL frameworks discussed in Section 7.

The EU AI Act [17] introduces the most structurally significant constraint. Cybersecurity tools deployed in critical infrastructure sectors are classified as high-risk systems under Annex III, triggering obligations for conformity assessment, transparency docu-

mentation, human oversight mechanisms, and demonstrable accuracy and robustness standards. The Act's transparency requirements map directly onto the explainability gaps identified in Section 6.1; its human oversight requirements formalize the HITL design gaps documented in Section 5.4. The European Banking Authority's 2025 factsheet on AI adoption in the EU banking and payments sector [92] identifies data governance, human oversight, consumer consent, and explainability as the leading challenges banks report when deploying AI, without yet prescribing a governance framework tailored to LLM-based systems—a gap this review's findings help characterize. Only one paper in the corpus, ThreatModeling-LLM [48], was developed explicitly for a regulated banking context, and even this work does not address AI Act conformity obligations. The gap between the regulatory requirements and the current state of the research literature is, at this point, substantial.

7. Open Research Directions and Future Work

Drawing on the domain-by-domain coverage gaps of Section 4, the architectural observations of Section 5, and the cross-cutting challenges of Section 6, we identify ten research directions that represent the most pressing open problems in LLM-based cybersecurity agents. These constitute the future work agenda we would prioritize, and Section 8 identifies the three we consider most consequential.

7.1. Standardized Orchestration Protocols for Multi-Agent Security Systems

Each multi-agent system in the corpus defines its own message schemas, memory structures, delegation semantics, and inter-agent trust models, making integration, reproducibility, and cross-system comparison difficult. Developing shared orchestration standards for security-oriented multi-agent systems—covering inter-agent communication formats, conflict resolution policies for contradictory subagent assessments, privilege tier specifications, and audit trail schemas—would allow specialist agents built by different teams to be composed into a single workflow with predictable security properties, without bespoke integration work at every boundary. MITRE ATT&CK [45] provides a precedent for community-driven standardization of adversary behavior knowledge; an analogous standard for agent coordination semantics would serve a similar function for the defensive tooling ecosystem. The "hallucinated compliance" failure mode documented by Nguyen and Husain [97] is, in part, a consequence of underspecified inter-agent trust contracts: formalizing these contracts is a prerequisite for production-safe deployment.

7.2. Incident Response and Forensics Agents: Filling the Critical Coverage Gap

Incident response and digital forensics represent the highest-value but most underdeveloped application area in the corpus: the IR category covers only five papers, and no paper addresses systematic digital forensics beyond log retrieval. This gap is particularly consequential because IR is the domain where LLM agents' multi-step reasoning, tool orchestration, and structured report generation capabilities are most directly applicable to time-pressured, high-stakes real-world workflows. MyAntFarm.ai's [37] 100% actionable response rate versus the 1.7% single-agent baseline demonstrates that orchestration design—not raw model capability—is the primary determinant of IR agent effectiveness. Future work should develop IR-agent architectures explicitly modeled on the NIST IR lifecycle phases (preparation; detection and analysis; containment, eradication, and recovery; post-incident activity), with output schemas designed to satisfy DORA's structured notification templates and NIS2's timeline obligations. The CORTEX system's 152.4 s median ticket resolution time [7] also establishes a performance baseline against which future systems

can be benchmarked, though it requires extension to multi-stage, multi-system incident scenarios before it can serve as a general evaluation framework.

7.3. Compliance-Aware Agent Design

Current agent architectures treat regulatory compliance as a documentation requirement appended at deployment time rather than a design constraint enforced at architecture time. We advocate for compliance-aware agent design as a first-class engineering paradigm: systems in which DORA reporting timelines [15], NIS2 notification obligations [16], and EU AI Act transparency requirements [17] are encoded as hard constraints on the agent's action space, output schemas, and audit mechanisms. Concretely, this entails action guards that require human confirmation before any decision requiring regulatory accountability; output schemas aligned with standardized notification templates such that regulatory filings can be generated directly from structured agent outputs; and immutable, cryptographically signed audit logs that satisfy both operational forensics requirements and legal discovery standards. The adoption challenges the European Banking Authority reports EU banks currently face—data governance, human oversight, consumer consent, and explainability [92]—map closely onto exactly these design requirements, suggesting a formal test harness for compliance-aware agent evaluation could be built directly around them.

7.4. Adaptive and Dynamic Benchmarking

The complete absence of E4-level evaluation in the corpus and the documented fragmentation of existing benchmarks both point to the same underlying need: a community-maintained, continuously updated benchmarking infrastructure for LLM cybersecurity agents. Static benchmark suites based on known CVE sets are inherently insufficient because the threat landscape evolves faster than academic benchmark refresh cycles, and because static data are susceptible to memorization by models trained on overlapping web corpora. AthenaBench [60] represents a step toward dynamic benchmarking within the CTI domain, but a general solution requires live test beds connected to real-time vulnerability intelligence feeds; adversarially constructed novel challenge samples that have not appeared in training corpora; and versioned snapshot releases to enable longitudinal performance tracking. A unified framework spanning all eight D1 security functions, with consistent metric definitions and open participation for industry operators who can contribute live incident data under appropriate data-sharing agreements, would substantially advance the field's ability to measure and communicate progress.

7.5. Federated and Privacy-Preserving Multi-Agent Collaboration

Collective defense—the ability of multiple organizations to contribute to and benefit from shared threat intelligence while retaining control over their sensitive data—is among the most effective mechanisms for improving sector-wide cyber resilience, particularly in financial services where common adversaries target many institutions simultaneously. Yet the data residency and confidentiality obligations documented in Section 6.4 make centralized cloud-based knowledge sharing infeasible for regulated institutions. Federated multi-agent architectures, in which specialized agents operate on locally siloed data while contributing privacy-preserving summaries or obfuscated model updates to shared orchestration layers, remain almost entirely unexplored in the corpus. Techniques from federated learning, differential privacy, and secure multi-party computation offer transferable methodological foundations. Demonstrating that a federated CTI network can match the threat detection capability of a centralized system while satisfying GDPR and data localization requirements would be a significant enabling result for the financial sector.

7.6. Formal Human-in-the-Loop Frameworks

Section 5.4 established that HITL posture is the most consequential safety-relevant design variable for production cybersecurity agents, yet no paper in the corpus provides a formal, reusable HITL framework. The four-component model proposed in Section 5.4—decision taxonomy, handoff interface, override mechanism, and audit trail—represents a minimal specification. A mature HITL framework must additionally address (a) cognitive load models ensuring that human reviewers are not overwhelmed by agent-generated evidence at volume; (b) formal representations of the information asymmetry between agent (which has processed large data volumes) and analyst (who must make a high-stakes decision on the basis of a summary); (c) feedback integration mechanisms enabling analyst corrections to propagate back into the agent's knowledge base and inform future behavior; and (d) formal verification methods capable of providing correctness guarantees for the oversight logic itself. Methodological foundations from human-robot interaction, autonomous vehicle safety assurance, and formal methods offer transferable techniques that have not yet been applied to cybersecurity agent oversight.

7.7. Multilingual Threat Intelligence

Every CTI system in the corpus is evaluated exclusively on English-language sources. This is a substantive limitation rather than a detail of experimental design. A large share of the raw material from which threat intelligence is derived—underground forum activity, ransomware negotiation transcripts, regional CERT advisories, and actor-authored tooling documentation—is not in English, and the languages involved are precisely those least well represented in the pre-training corpora of the backbones these systems rely on. Reported extraction accuracy should therefore be read as an upper bound that has not been shown to transfer.

Two research needs follow. The first is methodological: whether entity and relation extraction quality degrades gracefully or catastrophically across languages, and whether translation-then-extract pipelines preserve the fine-grained technical terminology on which TTP mapping depends. The second is infrastructural: no multilingual CTI benchmark currently exists, so the question cannot be answered comparably even by researchers who wish to.

7.8. Underserved Security Domains

Beyond the imbalances visible within the eight coded functions, four infrastructure-critical domains are effectively absent from the corpus. We note them here because their absence is unlikely to reflect a judgment that agents are unsuited to them.

Cloud and container security. Misconfiguration triage across IaC templates, Kubernetes policy, and cloud IAM surfaces is a multi-step, tool-mediated reasoning task of exactly the shape agents handle well, yet no corpus study addresses it.

Operational technology and industrial control systems. OT environments combine legacy protocols, low tolerance for active probing, and safety-critical consequences—a setting in which the HITL question is sharper than anywhere in the current corpus, and correspondingly under-examined.

Identity and access management. Entitlement review, privilege creep detection, and access recertification are high-volume analytical tasks currently performed manually at scale, with clear audit requirements that suit an agent's ability to produce traceable justifications.

Software supply chain and SBOM integrity. Dependency provenance, transitive vulnerability propagation, and build-system compromise require reasoning across heterogeneous artifacts and registries; the one corpus study touching this area treats it peripherally.

7.9. Re-Validation on Current-Generation Models

The corpus closes in April 2026, and 22 of 59 studies use a GPT-4-family backbone. Frontier model capability has moved considerably since the period in which most of these systems were built and evaluated, particularly on long-horizon planning and sustained tool use—the two capabilities that most directly bound agent performance in penetration testing and incident response.

This matters asymmetrically across our findings. The *architectural* results should prove relatively durable: the advantages of decomposition, specialization, and debate follow from the structure of the problem rather than from any specific model's ceiling, and MyAntFarm.ai's TinyLlama result already demonstrates that orchestration can substitute for scale. The *capability* results are far more perishable. Reported failure rates at planning and privilege escalation stages, and the observed gap between proprietary and open-weight backbones, are measurements of a particular model generation and should be re-run rather than cited forward. We would regard systematic re-evaluation of the corpus benchmarks on current-generation models as among the more urgent contributions available in this area, precisely because it is unglamorous and therefore unlikely to be done otherwise.

7.10. Financial Sector Specifics

A caveat must precede this subsection, because the framing is otherwise liable to be read as stronger than the evidence permits. The corpus contains a single compliance and risk assessment study, which is also its only banking-specific system. Everything that follows is therefore a *directional* argument grounded in regulatory requirements and in the observed absence of research, not a synthesis of empirical findings. The near-total absence of financial-sector work is itself the finding; the research agenda below is what we infer from it.

The financial sector presents a set of requirements that cross-domain cybersecurity agent research has not yet systematically addressed. Latency constraints for real-time transaction fraud detection and algorithmic trading anomaly detection are measured in milliseconds, whereas the agent response times documented in the corpus range from seconds to minutes. Explainability requirements for regulatory reporting under DORA and GDPR must satisfy legal standards that exceed what current reasoning trace outputs provide. Integration with financial data schemas—SWIFT message formats, ISO 20022 [100] payment standards, FIX protocol—requires domain knowledge absent from general-purpose training corpora. The threat model is also distinct: financially motivated APT actors, insider threats enabled by privileged access, and market manipulation vectors require specialized detection capabilities beyond the network-intrusion and malware-analysis focus of the majority of corpus papers.

ThreatModeling-LLM [48], the sole corpus paper developed explicitly for a banking context, demonstrates that domain-specific evaluation substantially changes the performance picture: mitigation-code accuracy improves from 0.36 to 0.69 (precision from 0.49 to 0.73) when prompt engineering and fine-tuning are combined. This represents a single data point, but it motivates a structured research programme specifically addressing financial-sector agent design, evaluation under DORA compliance constraints, and responsible deployment under the EU AI Act's high-risk system obligations. Such a programme would directly benefit the community of financial institutions that must implement these systems under binding regulatory timelines.

8. Conclusions

This paper has presented a PRISMA-guided systematic review of LLM-based agents in cybersecurity, synthesizing 59 primary studies published between January 2022 and April 2026

across eight security function categories. The review applied a five-dimensional taxonomy covering security function (D1), agent architecture (D2), knowledge augmentation (D3), human-in-the-loop posture (D4), and evaluation rigor (D5), enabling structured cross-paper comparison that prior narrative surveys have not provided.

Four primary findings emerge from the analysis. *First*, penetration testing dominates the corpus (50.8% of papers, a clear majority), reflecting both the well-defined task structure of offensive security and the high academic interest in autonomous capability demonstration, while incident response (8.5%), intrusion detection (5.1%), and compliance verification (1.7%) remain critically underrepresented relative to their operational importance. *Second*, multi-agent centralized hierarchical orchestration (MA-CH) is the strongest emerging architectural pattern, though not the most common one: it appears in 11 of 59 studies, against 18 for single-agent tool-calling, yet HPTSA achieves a 4.3× improvement over single-agent exploitation on zero-day targets, and MyAntFarm.ai's orchestrated 100% actionable IR rate versus 1.7% for single-agent approaches suggests that orchestration design can matter more than raw model scale. Both results rest on single systems, and the gap between MA-CH's effect sizes and its adoption is one of the clearer signals in the corpus of where effort is currently mismatched to payoff. *Third*, evaluation rigor remains insufficient across the field: no paper achieves E4-level production evaluation, PentestEval documents a 31% end-to-end task completion ceiling, and benchmark fragmentation renders cross-paper comparison unreliable. *Fourth*, GPT-4 dependence (22 of 59 studies, 37.3%) creates a capability monoculture that is architecturally incompatible with the privacy, data residency, and on-premises deployment requirements of regulated financial sector environments—though the year-on-year decline in that dependence, from 54.5% of 2024 studies to 15.4% of those dated 2026, suggests the obstacle is being dismantled by the field's own drift toward fine-tuned open-weight backbones rather than by any deliberate regulatory response.

The cross-cutting challenges of hallucination, prompt injection, benchmark fragmentation, data privacy, and regulatory compliance collectively define the distance between the current research frontier and production-ready deployment. The EU AI Act's classification of financial sector cybersecurity tools as high-risk systems, combined with DORA's binding incident reporting timelines, means this distance carries regulatory as well as technical consequences.

Limitations. Four constraints bound the strength of these conclusions. *Retrieval and completeness.* Records were identified through web-search-mediated retrieval rather than partitioned database queries (Section 3.1), and the resulting corpus is arXiv-weighted. Rather than leave recall unbounded, we measured it: an independent search of six bibliographic databases recovers 86.3% of the studies the query set had surfaced, with the shortfall clustering in venues those databases do not index. The same search indicates that the eligible literature is far larger than the corpus—of 828 plausibly in-scope records not already included, a random sample of 60 was 41.7% eligible (95% CI 29–54%), implying roughly 400 eligible studies in total. **This review therefore analyzes a documented subset, not a census.** We consider the subset adequate for its purpose, which is to populate and test a taxonomy rather than to enumerate a field growing faster than any single review can track; but conclusions about the *relative frequency* of patterns—the D1–D5 distributions in Table 4—should be read as descriptive of this corpus and treated as provisional for the field. Findings that rest on the absence of something, most importantly the absence of any production-grade (E4) evaluation, are correspondingly weaker: we can say no study in 59 reports one, not that none exists. Both the full query record (Appendix C) and the verification search (Appendix D) are published so that this can be reassessed. *Screening.* Screening was conducted by a single reviewer with an AI-assisted consistency pass. No second reviewer re-screened a sample, so no inter-rater agreement statistic is available;

the protocol was not registered prospectively with PROSPERO or OSF, and no formal risk-of-bias instrument was applied beyond the E1–E4 coding, which measures evaluation strength rather than risk of bias. We publish the complete per-record screening decision set instead, so that the screening can be audited rather than taken on trust. In the deposited screening record, the exclusion codings were generated by an AI-assisted pass and adopted by the authors rather than verified record by record (Section 3.2), and the individual reasons for some full-text exclusions were not retained. Readers who need exclusion-level certainty should treat those codings as indicative and consult the per-record evidence in the deposit. *Recency.* The April 2026 cut-off means the capability findings in particular reflect a specific model generation and should be re-run rather than cited forward (Section 7.9). *Financial-sector evidence.* The regulatory analysis rests on a single paper that is simultaneously the corpus's only compliance study and its only banking-specific system; those conclusions are directional and are presented as such.

LLM-based agents are a substantive addition to the cybersecurity toolkit: the architecture, knowledge augmentation, and tool integration patterns documented here make possible classes of autonomous, multi-step security reasoning that were not achievable before 2022. The field is, however, still in an early phase characterized by proof-of-concept systems evaluated in controlled laboratory settings, and the single most telling statistic in this review is that not one of 59 studies reports a production deployment. Closing that gap requires concurrent progress on evaluation infrastructure, adversarial robustness, HITL design, and regulatory alignment. Section 7 sets out ten directions for future work, of which we would single out three as most consequential: filling the incident response and forensics gap, where operational need and research attention are furthest apart; building adaptive benchmarks to replace the static, contamination-prone test beds the field currently relies on; and re-validating the capability findings on current-generation models. We hope the taxonomy, the published coding matrix, and the gap analysis presented here provide a useful foundation for that work.

Supplementary Materials: The following supporting information can be downloaded at https://www.mdpi.com/article/10.3390/jcp6050159/s1: Table S1, the PRISMA 2020 Checklist used in the current systematic review study. The review data and analysis code underlying this review—the review protocol and coding manual; the search record for both the primary retrieval and the independent verification search, with the raw database exports; per-record screening decisions for all 211 records entering screening; the classification of the 59 included studies; the eligibility sample underlying the population estimate in Section 3.1; and the scripts that regenerate Tables 4 and 6 from the classification and validate the screening record against the PRISMA flow—are openly deposited at Zenodo; see the Data Availability Statement.

Author Contributions: Conceptualization, G.F. and J.S.; methodology, G.F.; investigation, G.F. and G.M.; writing—original draft preparation, G.F.; writing—review and editing, G.F., K.M., G.M., J.S. and D.K.; visualization, G.F.; supervision, J.S. and D.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the European Union under the Digital Europe Programme, call DIGITAL-ECCC-2024-DEPLOY-CYBER-07, grant agreement no. 101249596 (CyberAId project). The APC was funded by the European Union under the Digital Europe Programme, grant agreement no. 101249596 (CyberAId project).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All materials underlying this review are openly available at Zenodo, https://doi.org/10.5281/zenodo.22025510, and an identical copy is included with this article as Supplementary Materials: the complete search record for both the primary retrieval (59 queries with per-query yields and retrieved titles) and the independent verification search (six databases, with raw exports, execution dates and per-database yields); per-record screening decisions for all 211 records entering screening, giving the criterion applied and the basis of each decision; the review protocol and coding manual; the full classification of the 59 included studies reproduced as Appendix B; the eligibility sample underlying the population estimate in Section 3.1; and the scripts that generate Tables 4 and 6 from the classification and validate them against the figures reported here. The Zenodo record is the citable and versioned copy: any correction to the coding that proves necessary after publication will be issued there as a new version under the same concept DOI. No new empirical data were generated by the authors; further inquiries can be directed to the corresponding author.

Acknowledgments: The authors acknowledge the support of the CyberAId consortium partners. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or the European Cybersecurity Competence Center (ECCC). During the preparation of this study, the authors used Anthropic Claude (Claude 3.5 Sonnet, accessed April 2026) for the purpose of an AI-assisted record screening pass applying the pre-specified inclusion and exclusion criteria, as described in Section 3.2. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: Authors George Fatouros and John Soldatos were employed by the company Innov-Acts Ltd., a consortium partner in the EU-funded CyberAId project (Grant Agreement No. 101249596). The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as potential conflicts of interest. The authors declare that this study received funding from the European Union under the Horizon Europe programme, grant number 101249596 (CyberAId). The funder was not involved in the study design, collection, analysis, interpretation of data, the writing of this article or the decision to submit it for publication.

Abbreviations

The following abbreviations are used in this manuscript:

APT	Advanced Persistent Threat
CI/CD	Continuous Integration/Continuous Delivery
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DORA	Digital Operational Resilience Act
GA	Grouping Accuracy
HITL	Human-in-the-Loop
HYB	Hybrid knowledge augmentation
IaC	Infrastructure as Code
ICL	In-Context Learning
IDS	Intrusion Detection System
IOCs	Indicators of Compromise
IR	Incident Response
KG	Knowledge Graph
LLM	Large Language Model
MA	Multi-Agent
MTTD	Mean Time to Detect
MTTM	Mean Time to Mitigate
NIS2	Network and Information Security Directive 2
NLP	Natural Language Processing

NVD	National Vulnerability Database
OWASP	Open Web Application Security Project
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
PT	Penetration Testing
RAG	Retrieval-Augmented Generation
SA	Single Agent
SAST	Static Application Security Testing
SBOM	Software Bill of Materials
SCA	Software Composition Analysis
SIEM	Security Information and Event Management
SMTP	Simple Mail Transfer Protocol
SOC	Security Operations Center
TTP	Tactics, Techniques, and Procedures
VA	Vulnerability Assessment
XDR	Extended Detection and Response

Appendix A. PRISMA 2020 Checklist

The completed PRISMA 2020 checklist, mapping each reporting item to its location in this manuscript, is provided in the Supplementary Materials. Items relating to protocol registration and risk-of-bias assessment are marked as not applicable, with the reasons stated in Sections 3 and 8.

Appendix B. Master Classification Table

Table A1. Full per-paper classification across all five taxonomy dimensions. Papers are grouped by primary security function (D1) and sorted by year. Multi-function papers show the secondary domain in parentheses; the secondary code is recorded here but excluded from the marginal counts in Table 4. The *Primary model* column records the backbone used for each study's headline result; where a study evaluates several backbones, the one reported in its principal result is given, and "Multiple" indicates a study with no single headline configuration. Columns D1–D5 are the five taxonomy dimensions defined in Table 3. *Year* is the year of the version cited: several studies were retrieved as preprints inside the January 2022–April 2026 search window and are cited here in their subsequently published form, which in some cases carries a later date.

Ref.	System	Yr.	Primary Model	D1	D2	D3	D4	D5
Threat Intelligence (TI) ($n = 8$)								
[56]	Crimson	2024	Llama-2-7B (FT)	TI	SA-IO	HYB	FA	E2
[54]	CTIKG	2024	Yi-34B-4bits	TI	MA-PP	RAG	FA	E2
[58]	Tseng et al.	2024	GPT-4	TI	SA-IO	RAG	FA	E2
[55]	CTINexus	2025	GPT-4	TI	SA-IO	ICL	FA	E2
[57]	Proactive TI	2025	GPT-4o	TI	SA-IO	RAG	FA	E1
[26]	CTI-Thinker	2026	LLaMA-3-8B (FT)	TI	SA-IO	HYB	FA	E2
[61]	OntoLogX	2026	Multiple	TI	SA-IO	RAG	FA	E3
[59]	TTPDetect	2026	Claude 3.7 Sonnet	TI	SA-TC	RAG	FA	E2
Vulnerability Assessment (VA) ($n = 6$)								
[62]	ChatAFL	2024	GPT-3.5-Turbo	VA	SA-IO	ICL	FA	E3
[30]	FuzzGPT	2024	Codex	VA	SA-IO	ICL	FA	E3
[24]	IRIS	2024	GPT-4	VA	SA-IO	ICL	FA	E3
[63]	LLMDFA	2024	GPT-3.5	VA	SA-IO	ICL	FA	E3
[65]	MultiFuzz	2025	Multiple	VA	MA-PP	RAG	FA	E3
[64]	VulSolver	2025	Kimi-K2	VA	SA-IO	ICL	FA	E3
DevSecOps and SOC Automation (DS) ($n = 3$)								
[67]	CodeQL Repair Study	2025	Multiple	DS	SA-IO	ICL	FA	E3
[7]	CORTEX	2025	GPT-5-mini	DS	MA-CH	RAG	FA	E3
[47]	CyberRAG	2025	Llama-3.1-8B	DS (ID)	SA-TC	HYB	FA	E3

Table A1. *Cont.*

Ref.	System	Yr.	Primary Model	D1	D2	D3	D4	D5
Intrusion Detection (ID) ($n = 3$)								
[39]	Audit-LLM	2024	GPT-3.5-Turbo-0125	ID	MA-DB	ICL	FA	E3
[32]	IDS-Agent	2024	GPT-4o	ID	SA-TC	RAG	FA	E3
[52]	Sahay et al.	2026	ChatGPT	ID (DS)	MA-CH	ICL	HA	E2
Penetration Testing and Red Teaming (PT) ($n = 30$)								
[68]	HackingBuddyGPT	2023	GPT-3.5-turbo	PT	SA-TC	ICL	FA	E2
[96]	AutoAttacker	2024	GPT-4	PT	SA-TC	RAG	FA	E3
[101]	AutoPT	2024	GPT-4o mini	PT	SA-PE	ICL	FA	E3
[102]	Enigma	2024	Claude 3.5 Sonnet	PT	SA-TC	ICL	FA	E3
[71]	Fang 1-day	2024	GPT-4	PT	SA-TC	ICL	FA	E3
[70]	Fang Websites	2024	GPT-4	PT	SA-TC	RAG	FA	E3
[76]	HackSynth	2024	GPT-4o	PT	SA-TC	ICL	FA	E2
[83]	NYU CTF Bench	2024	GPT-4	PT	SA-TC	ICL	FA	E3
[87]	Pentest Copilot	2024	GPT-4-Turbo	PT	SA-TC	RAG	HG	E2
[10]	PentestGPT	2024	GPT-4	PT	MA-CH	ICL	HG	E3
[95]	AutoPentester	2025	GPT-4-turbo	PT (VA)	MA-CH	RAG	HG	E3
[74]	CheckMate	2025	Sonnet 4.5	PT	SA-IO	ICL	FA	E3
[82]	CTF Multi-Agent	2025	GPT-4o	PT	MA-CH	ICL	HG	E2
[88]	MalGEN	2025	Qwen2.5 Coder-3B	PT	MA-PP	ICL	FA	E3
[97]	Nguyen and Husain	2025	Multiple	PT	MA-CH	ICL	FA	E3
[85]	Pentest Arch. Study	2025	Multiple	PT	MA-CH	ICL	HG	E3
[79]	Pentest-R1	2025	DeepSeek-R1-0528-Qwen3-8B (FT)	PT	SA-TC	FT	FA	E3
[103]	PentestAgent	2025	GPT-4	PT	MA-PP	RAG	FA	E3
[81]	Random-Crypto RL	2025	Llama-3.1-8B (FT)	PT	SA-TC	FT	FA	E3
[77]	RedTeamLLM	2025	GPT-4o	PT	SA-TC	ICL	FA	E3
[36]	VulnBot	2025	Llama3.1-405B	PT	MA-PP	HYB	FA	E3
[78]	xOffense	2025	Qwen3-32B (FT)	PT	MA-CH	HYB	FA	E3
[33]	CTFAgent	2026	Multiple	PT	SA-PE	ICL	FA	E3
[84]	CTFExplorer	2026	Multiple	PT	MA-CH	ICL	FA	E3
[80]	Cyber-Zero	2026	Qwen3-32B (FT)	PT	SA-TC	FT	FA	E3
[72]	HPTSA	2026	GPT-4-0125-preview	PT	MA-CH	ICL	FA	E3
[69]	LLMs as Hackers	2026	GPT-4-Turbo	PT	SA-TC	ICL	FA	E3
[73]	PenForge	2026	Claude-3.7-Sonnet-20250219	PT	MA-CH	RAG	FA	E3
[34]	PentestGPT v2	2026	Claude Opus 4.5	PT	SA-PE	RAG	FA	E3
[86]	ROS Pentest Workflow	2026	llama-3.3-70b-instruct	PT	MA-PP	RAG	HG	E3
Incident Response and Forensics (IR) ($n = 5$)								
[90]	Liu IR	2024	GPT-4o	IR	MA-DB	ICL	FA	E3
[89]	Novel Defense	2024	Multiple	IR (ID)	SA-TC	ICL	FA	E3
[37]	MyAntFarm.ai	2025	TinyLlama-1B	IR	MA-PP	ICL	FA	E2
[53]	Tellache et al.	2025	GPT-4o	IR (TI)	SA-IO	RAG	FA	E2
[91]	AIR	2026	GPT-5	IR	SA-TC	ICL	FA	E3
Compliance and Risk Assessment (CR) ($n = 1$)								
[48]	ThreatModeling-LLM	2024	Llama-3.1-8B (FT)	CR (VA)	SA-IO	HYB	FA	E2
Phishing and Social Engineering (PH) ($n = 3$)								
[93]	KnowPhish	2024	GPT-3.5-turbo-instruct	PH	SA-IO	HYB	FA	E3
[94]	MultiPhishGuard	2025	GPT-4o	PH	MA-PP	ICL	FA	E3
[40]	PhishDebate	2025	Multiple	PH	MA-DB	ICL	FA	E3

Appendix C. Search Record

This appendix reproduces the complete retrieval record so that the identification stage of the review can be repeated and its coverage independently assessed. Table A2 lists all 59 queries verbatim, in execution order. Queries were executed between 8 and 22 April 2026; as noted in Section 3.1, only the first page of results (at most ten) was assessed per query, so per-query result counts are not separately tabulated below—they would report the pagination cap, not the yield of each query. The 584 total results are reported in Section 3.3.

Queries are grouped below by the concept cluster they principally address (Section 3.1), but the grouping is descriptive rather than procedural: several queries span clusters, and named-system lookups were issued opportunistically as candidate systems surfaced in

earlier results. This iterative, snowballing character is a property of web-search-mediated retrieval and is reported rather than concealed.

Table A2. Complete search record: 59 queries in execution order, reproduced verbatim as executed.

No.	Query as Executed
C1: Core agent and security; seed surveys and landmark systems	
1	site:arxiv.org ‘‘LLM agent’’ cybersecurity survey
2	large language model agent security survey arxiv
3	LLM agent cybersecurity survey paper 2024 2025
4	CyberSecEval Meta LLM security benchmark paper 2024
5	AutoAttacker LLM automated cyberattack paper arxiv 2024
6	PentestGPT paper arxiv Deng 2024 penetration testing LLM
7	Cybench SecBench ‘‘LLM agent’’ cybersecurity benchmark Happe Cito HackingBuddyGPT
8	Fang ‘‘LLM agents can autonomously exploit’’ one-day vulnerabilities arxiv 2024
9	‘‘Teams of LLM Agents’’ ‘‘zero-day vulnerabilities’’ Fang Kang 2024 arxiv
10	SecGPT LLM agent cybersecurity incident response threat intelligence 2024
C2: Security functions	
11	‘‘LLM SOC automation’’ agent
12	‘‘LLM incident response’’ agent
13	‘‘LLM threat intelligence’’ agent
14	‘‘LLM intrusion detection’’ agent
15	site:arxiv.org ‘‘LLM’’ ‘‘malware analysis’’ agent 2024 2025
16	‘‘LLM’’ ‘‘alert triage’’ SOC agent 2024
17	‘‘LLM’’ ‘‘phishing detection’’ agent 2024 2025
18	‘‘LLM’’ ‘‘threat hunting’’ ‘‘log analysis’’ security agent
19	KnowPhish USENIX Security 2024 LLM phishing detection
20	‘‘CTIBench’’ benchmark LLM cyber threat intelligence 2024
21	‘‘SEvenLLM’’ OR ‘‘SecureBERT’’ OR ‘‘SecGPT’’ fine-tuned LLM cybersecurity defense 2024
22	‘‘Crimson’’ LLM CVE MITRE ATT&CK threat intelligence 2024
23	‘‘Audit-LLM’’ insider threat detection multi-agent 2024 arxiv
24	‘‘TTPDetect’’ LLM malware binary MITRE ATT&CK agent 2025
C4: Financial sector, compliance, and regulation	
25	‘‘LLM’’ ‘‘financial cybersecurity’’ agent
26	‘‘LLM’’ ‘‘fraud detection’’ agentic agent
27	‘‘large language model’’ ‘‘financial sector’’ cybersecurity agent
28	‘‘LLM DevSecOps’’ agent ‘‘secure code review’’
29	‘‘LLM’’ ‘‘GRC’’ governance risk compliance agent automation
30	‘‘LLM’’ ‘‘SBOM’’ ‘‘supply chain security’’ software agent
31	Gartner Forrester LLM agents financial security 2024 2025
32	‘‘LLM’’ ‘‘ISO 27001’’ OR ‘‘NIST’’ compliance automation agent cybersecurity paper
33	‘‘EU AI Act’’ ‘‘financial services’’ ‘‘security agents’’ LLM implications 2024 2025
34	arxiv ‘‘LLM agent’’ ‘‘threat intelligence’’ ‘‘incident response’’ security operations center 2024 2025
35	‘‘LLM’’ ‘‘Copilot’’ ‘‘secure code’’ vulnerability detection agent 2024 2025 paper
36	arxiv ‘‘ThreatModeling-LLM’’ banking threat modeling NIST 800-53 Yang 2024
37	‘‘SEVENLLM’’ OR ‘‘SecureFalcon’’ OR ‘‘Audit-LLM’’ OR ‘‘HuntGPT’’ financial cybersecurity LLM agent paper 2024
C3: Agent architecture and orchestration	
38	site:arxiv.org multi-agent LLM cybersecurity 2024 2025
39	‘‘LangGraph’’ security agent cybersecurity
40	‘‘ReAct agent’’ security cybersecurity LLM
41	‘‘multi-agent orchestration’’ cybersecurity LLM
42	PentestGPT LLM agent penetration testing arxiv
43	‘‘AutoGen’’ cybersecurity agent penetration testing
44	‘‘retrieval augmented generation’’ cybersecurity CVE MITRE ATT&CK
45	‘‘CyberRAG’’ OR ‘‘SecGPT’’ OR ‘‘CybersecurityGPT’’ LLM agent framework arxiv
46	VulnBot multi-agent penetration testing arxiv 2025
47	‘‘cooperative agents’’ OR ‘‘debate’’ multi-agent security LLM SOC incident response
48	‘‘LLM’’ ‘‘knowledge graph’’ cybersecurity agent threat intelligence arxiv

Table A2. *Cont.*

#	Query as Executed
C2/C3: Penetration testing, vulnerability discovery, and benchmarks	
49	`site:arxiv.org ‘‘LLM’’ ‘‘penetration testing’’ 2024 2025`
50	`‘‘PentestGPT’’ OR ‘‘AutoPentest’’ OR ‘‘PENTESTGPT’’`
51	`‘‘LLM penetration testing’’ agent`
52	`‘‘LLM vulnerability detection’’ code agent 2024 2025`
53	`‘‘LLM’’ ‘‘capture the flag’’ CTF cybersecurity agent benchmark`
54	`‘‘LLM red teaming’’ cybersecurity agent automated 2024`
55	`‘‘Cybench’’ OR ‘‘EnIGMA’’ LLM cybersecurity agent benchmark CTF`
56	`‘‘LLM fuzzing’’ ‘‘ChatAFL’’ OR ‘‘FuzzGPT’’ security vulnerability`
57	`‘‘AutoAttacker’’ LLM post-breach attack agent arxiv`
58	`‘‘language agents as hackers’’ OR ‘‘HackSynth’’ OR ‘‘AutoPT’’ LLM cybersecurity arxiv`
59	`site:arxiv.org ‘‘LLMDFA’’ OR ‘‘VulDetectBench’’ OR ‘‘LLMs Cannot Reliably’’ vulnerability detection`

Of the 584 results returned, 520 carried an identifiable bibliographic host. Table A3 reports their distribution. Two features of this distribution are worth stating plainly. First, arXiv accounts for the large majority of scholarly results, which reflects both the genuine preprint concentration of this literature and the ranking behavior of the search engine; it should not be read as evidence that other indexes were searched to comparable depth. Second, roughly 45% of returned results were non-scholarly—vendor blog posts, code repositories, trade press, and analyst notes—and were removed before screening. A per-database query protocol would not have returned these, and their presence is a direct consequence of the retrieval method.

Table A3. Distribution of the 520 source-attributable results across bibliographic hosts, before deduplication. Hosts returning fewer than four results are aggregated.

Host	Results	% of Attributable
arXiv	238	45.8
GitHub/GitHub Pages	43	8.3
MDPI	16	3.1
Medium	16	3.1
ACM Digital Library	9	1.7
Elsevier ScienceDirect	8	1.5
USENIX	8	1.5
SpringerLink	8	1.5
ResearchGate	8	1.5
MITRE	6	1.2
Emergent Mind	6	1.2
Semantic Scholar	5	1.0
Gartner	5	1.0
ACL Anthology	4	0.8
IEEE Xplore	1	0.2
Other hosts (<4 results each)	139	26.7
Total	**520**	**100.0**

Title-level deduplication reduced the 520 attributable results to 380 unique records; removal of non-scholarly items left 174 unique scholarly records, to which citation chaining added 37, giving the 211 records that entered screening (Figure 2).

Appendix D. Verification Search

This appendix reports the independent database search used to characterize the coverage of the primary retrieval (Section 3.1). The exact Boolean string executed against each interface, adapted to its syntax, is published with the data deposit; Table A4 summarizes the sources, execution dates, and yields. Where an interface imposed limits—ScienceDirect permits at most eight Boolean connectors, IEEE Xplore caps citation export at 100 records per file—the query or the export was split accordingly.

Table A4. Verification search: Sources, execution dates, and yields.

Source	Queries	Executed	Hits	Exported
arXiv	8	12 August 2026	982	761
Scopus	1	13 August 2026	505	505
IEEE Xplore	1	12 August 2026	416	416
SpringerLink	1	12 August 2026	932	932
ACM Digital Library	1	12 August 2026	43	43
Elsevier ScienceDirect	1	12 August 2026	25	25
Total	**16**		**2903**	**2682**

After title-level deduplication, the six sources yield 2331 unique records. Web of Science and Semantic Scholar were not pursued: the marginal contribution of additional sources had already reached zero, in that the ACM Digital Library and ScienceDirect each recover no included study that another source does not also return.

Recovery. Matching was performed on DOI first, then arXiv identifier, then normalized title, and then distinctive system name; identifier-based matching is necessary here because several preprints in the corpus have been retitled in later versions. The searches recover 47 of the 59 included studies (79.7%), and 44 of the 51 that the primary retrieval had surfaced (86.3%). Table A5 lists those not recovered.

Table A5. Included studies not returned by the verification search. "Chained" marks studies identified by citation chaining rather than by the query set, which are not expected to appear.

Study	Venue or Reason Not Indexed	Chained
CTIKG [54]	Conference on Language Modeling	yes
ChatAFL [62]	NDSS	yes
FuzzGPT [30]	ICSE	yes
LLMDFA [63]	NeurIPS	yes
IDS-Agent [32]	NeurIPS workshop	yes
KnowPhish [93]	USENIX Security—not indexed by any source searched	no
Crimson [56]	small regional conference	no
Novel Defense [89]	arXiv preprint outside the executed strings	no
Audit-LLM [39]	arXiv preprint outside the executed strings	no
Enigma [102]	arXiv preprint outside the executed strings	no
ThreatModeling-LLM [48]	arXiv preprint outside the executed strings	no
MalGEN [88]	arXiv preprint outside the executed strings	no

Sensitivity. Leave-one-out analysis over the six sources shows the result is dominated by arXiv: removing it reduces recovery from 86.3% to 36.5%, and arXiv alone recovers 78.8%. Removing Scopus costs two studies, IEEE Xplore and SpringerLink one each, and the ACM Digital Library or ScienceDirect none. This is a quantitative statement of the same property that motivated the primary retrieval strategy: as of the search window, this literature is disseminated first and predominantly as preprints.

Scale of the eligible literature. The verification search returned 2283 records not matched to the corpus, of which 828 mention both an agent or LLM term and a security

term. A random sample of 60 of those 828 was screened against the criteria of Section 3.2: 25 were eligible, a rate of 41.7% (95% CI 29–54%), with the exclusions dominated by systems using an LLM as a classifier with no agentic loop (EC1, 14 records) and by work on the security of LLMs rather than LLMs for security (EC2, 8 records). Extrapolated, this implies approximately 345 further eligible studies (CI 242–448) within these six databases, and an eligible literature of roughly 400 studies in total. The sample and its per-record decisions are published with the deposit. This estimate is the basis for the statement in Section 8 that the corpus is a documented subset rather than a census.

References

1. Srinivas, S.; Kirk, B.; Zendejas, J.; Espino, M.; Boskovich, M.; Bari, A.; Dajani, K.; Alzahrani, N. AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation. *J. Cybersecur. Priv.* **2025**, *5*, 95. [CrossRef]
2. IBM Security. *Cost of a Data Breach Report 2024*; Technical Report; IBM: Armonk, NY, USA, 2024.
3. Xu, F.; Hao, Q.; Shao, C.; Zong, Z.; Li, Y.; Wang, J.; Zhang, Y.; Wang, J.; Lan, X.; Gong, J.; et al. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns* **2025**, *6*, 101370. [CrossRef] [PubMed]
4. Fatouros, G.; Metaxas, K.; Soldatos, J.; Karathanassis, M. MarketSenseAI 2.0: Enhancing Stock Analysis Through LLM Agents. In Proceedings of the 2025 IEEE International Conference on Data Mining Workshops (ICDMW), Washington, DC, USA, 12–15 November 2025; pp. 883–892. [CrossRef]
5. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR 2023), Kigali, Rwanda, 1–5 May 2023.
6. Wang, L.; Ma, C.; Feng, X.; Zhang, Z.; Yang, H.; Zhang, J.; Chen, Z.; Tang, J.; Chen, X.; Lin, Y.; et al. A Survey on Large Language Model Based Autonomous Agents. *Front. Comput. Sci.* **2024**, *18*, 186345. [CrossRef]
7. Wei, B.; Tay, Y.S.; Liu, H.; Pan, J.; Luo, K.; Zhu, Z.; Jordan, C. CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage. *arXiv* **2025**, arXiv:2510.00311. [CrossRef]
8. He, F.; Zhu, T.; Ye, D.; Liu, B.; Zhou, W.; Yu, P.S. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. *ACM Comput. Surv.* **2025**, *58*, 1–36. [CrossRef]
9. Kaniewski, S.; Schmidt, F.; Enzweiler, M.; Menth, M.; Heer, T. A Systematic Literature Review on Detecting Software Vulnerabilities with Large Language Models. *arXiv* **2025**, arXiv:2507.22659. [CrossRef]
10. Deng, G.; Liu, Y.; Mayoral-Vilches, V.; Liu, P.; Li, Y.; Xu, Y.; Zhang, T.; Liu, Y.; Pinzger, M.; Rass, S. PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing. In Proceedings of the 33rd USENIX Security Symposium, Philadelphia, PA, USA, 14–16 August 2024; pp. 847–864.
11. Shahriar, A.; Rahman, M.N.; Ahmed, S.; Sadeque, F.; Parvez, M.R. A Survey on Agentic Security: Applications, Threats and Defenses. *arXiv* **2025**, arXiv:2510.06445. [CrossRef]
12. Lazer, S.J.; Aryal, K.; Gupta, M.; Bertino, E. A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-case Prototypes. *arXiv* **2026**, arXiv:2601.05293. [CrossRef]
13. Tang, Y.; Liu, Y.; Lan, J.; Yan, Z.; Gelenbe, E. Security of LLM-based Agents Regarding Attacks, Defenses, and Applications: A Comprehensive Survey. *Inf. Fusion* **2025**, *127*, 103941. [CrossRef]
14. Xu, M.; Fan, J.; Huang, X.; Zhou, C.; Kang, J.; Niyato, D.; Mao, S.; Han, Z.; Shen, X.; Lam, K.Y. Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks. *arXiv* **2025**, arXiv:2505.12786. [CrossRef]
15. European Parliament and Council. *Regulation (EU) 2022/2554 on Digital Operational Resilience for the Financial Sector (DORA)*; Official Journal of the European Union; EU: Brussels, Belgium, 2022; Volume 333, pp. 1–79.
16. European Parliament and Council. *Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on Measures for a High Common Level of Cybersecurity Across the Union, Amending Regulation (EU) No 910/2014 and Directive (EU) 2018/1972, and Repealing Directive (EU) 2016/1148 (NIS 2 Directive)*; Official Journal of the European Union; EU: Brussels, Belgium, 2022; Volume 333, pp.80–135.
17. European Parliament and Council. *Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence (AI Act)*; Official Journal of the European Union 2024/1689; EU: Brussels, Belgium, 2024. Available online: http://data.europa.eu/eli/reg/2024/1689/oj (accessed on 5 July 2026).
18. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17), Long Beach, CA, USA, 4–9 December 2017*; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 6000–6010.
19. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 27730–27744. [CrossRef]

20. Rafailov, R.; Sharma, A.; Mitchell, E.; Manning, C.D.; Ermon, S.; Finn, C. Direct preference optimization: Your language model is secretly a reward model. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 53728–53741. [CrossRef]
21. Zhong, A.; Mo, D.; Liu, G.; Liu, J.; Lu, Q.; Zhou, Q.; Wu, J.; Li, Q.; Wen, Q. Logparser-llm: Advancing efficient log parsing with large language models. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Barcelona, Spain, 25–29 August 2024; pp. 4559–4570.
22. Xiao, Y.; Le, V.H.; Zhang, H. free: Towards more practical log parsing with large language models. In Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, Sacramento, CA, USA, 27 October–1 November 2024; pp. 153–165.
23. Boffa, M.; Drago, I.; Mellia, M.; Vassio, L.; Giordano, D.; Valentim, R.; Houidi, Z.B. LogPrécis: Unleashing language models for automated malicious log analysis: Précis: A concise summary of essential points, statements, or facts. *Comput. Secur.* **2024**, *141*, 103805.
24. Li, Z.; Dutta, S.; Naik, M. IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. *arXiv* **2024**, arXiv:2405.17238. [CrossRef]
25. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837. [CrossRef]
26. Yang, X.; Zhong, R.; Chen, Y.; Peng, G.; Yao, D.; Chen, C.; Wang, C.; Zhang, D.; Zhou, Y.; Yang, Z. CTI-Thinker: An LLM-Driven System for CTI Knowledge Graph Construction and Attack Reasoning. *Cybersecurity* **2026**, *9*, 106. [CrossRef]
27. Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 68539–68551. [CrossRef]
28. Alhuzali, A. LLM-Powered Threat Intelligence: A Retrieval-Augmented Generation Approach for Cyber Attack Investigation. *PeerJ Comput. Sci.* **2025**, *11*, e3371. [CrossRef]
29. Ferrag, M.A.; Tihanyi, N.; Hamouda, D.; Maglaras, L.; Lakas, A.; Debbah, M. From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows. *ICT Express* **2025**, *12*, 353–383. [CrossRef]
30. Deng, Y.; Xia, C.S.; Yang, C.; Zhang, S.D.; Yang, S.; Zhang, L. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, Lisbon, Portugal, 14–20 April 2024; pp. 1–13.
31. Alam, M.T.; Bhusal, D.; Nguyen, L.; Rastogi, N. CTIBench: A Benchmark for Evaluating LLMs in Cyber Threat Intelligence. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track, Vancouver, BC, Canada, 9–15 December 2024.
32. Li, Y.; Xiang, Z.; Bastian, N.D.; Song, D.; Li, B. IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks. In Proceedings of the NeurIPS 2024 Workshop on Open-World Agents: Synnergizing Reasoning and Decision-Making in Open-World Environments (OWA-2024), Vancouver, BC, Canada, 9–15 December 2024.
33. Zou, Y.; Liu, J.; Fan, W. Ctfagent: An llm-powered agent for ctf challenge solving. *J. Inf. Secur. Appl.* **2026**, *96*, 104305. [CrossRef]
34. Deng, G.; Liu, Y.; Li, Y.; Yang, R.; Xie, X.; Zhang, J.; Qiu, H.; Zhang, T. What Makes a Good LLM Agent for Real-world Penetration Testing? *arXiv* **2026**, arXiv:2602.17622. [CrossRef]
35. Moore, D.J. A Taxonomy of Hierarchical Multi-Agent Systems: Design Patterns, Coordination Mechanisms, and Industrial Applications. *arXiv* **2025**, arXiv:2508.12683. [CrossRef]
36. Kong, H.; Hu, D.; Ge, J.; Li, L.; Li, T.; Wu, B. VulnBot: Autonomous Penetration Testing for a Multi-Agent Collaborative Framework. *arXiv* **2025**, arXiv:2501.13411. [CrossRef]
37. Drammeh, P. Multi-Agent LLM Orchestration Achieves Deterministic, High-Quality Decision Support for Incident Response. *arXiv* **2025**, arXiv:2511.15755. [CrossRef]
38. Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J.B.; Mordatch, I. Improving factuality and reasoning in language models through multiagent debate. In Proceedings of the Forty-First International Conference on Machine Learning, Vienna, Austria, 21–27 July 2024.
39. Song, C.; Ma, L.; Zheng, J.; Liao, J.; Kuang, H.; Yang, L. Audit-LLM: Multi-Agent Collaboration for Log-Based Insider Threat Detection. *arXiv* **2024**, arXiv:2408.08902. [CrossRef]
40. Li, W.; Manickam, S.; Chong, Y.w.; Karuppayah, S. PhishDebate: An LLM-based multi-agent framework for phishing website detection. *arXiv* **2025**, arXiv:2506.15656. [CrossRef]
41. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
42. Chase, H. LangChain, Python Library for Building Applications with Large Language Models. 2023. Available online: https://github.com/langchain-ai/langchain (accessed on 8 August 2026).
43. Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In Proceedings of the First Conference on Language Modeling, Philadelphia, PA, USA, 7–9 October 2024.

44. National Institute of Standards and Technology. *Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1*; NIST Cybersecurity White Paper NIST CSWP 6; National Institute of Standards and Technology: Gaithersburg, MD, USA, 2018. [CrossRef]
45. MITRE Corporation. MITRE ATT&CK Enterprise Matrix, Version 14. 2023. Available online: https://attack.mitre.org (accessed on 1 April 2026).
46. *ISO/IEC 27001:2022*; Information Security, Cybersecurity and Privacy Protection—Information Security Management Systems—Requirements. International Organization for Standardization: Geneva, Switzerland, 2022.
47. Blefari, F.; Cosentino, C.; Pironti, F.A.; Furfaro, A.; Marozzo, F. CyberRAG: An agentic RAG cyber attack classification and reporting tool. *Future Gener. Comput. Syst.* **2025**, *176*, 108186. [CrossRef]
48. Wu, T.; Yang, S.; Liu, S.; Nguyen, D.; Jang, S.; Abuadbba, A. ThreatModeling-LLM: Automating threat modeling using large language models for banking system. *arXiv* **2024**, arXiv:2411.17058. [CrossRef]
49. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. [CrossRef] [PubMed]
50. Nykvist, B.; Macura, B.; Xylia, M.; Olsson, E. Testing the utility of GPT for title and abstract screening in environmental systematic evidence synthesis. *Environ. Evid.* **2025**, *14*, 7. [CrossRef] [PubMed]
51. Delgado-Chaves, F.M.; Jennings, M.J.; Atalaia, A.; Wolff, J.; Horvath, R.; Mamdouh, Z.M.; Baumbach, J.; Baumbach, L. Transforming literature screening: The emerging role of large language models in systematic reviews. *Proc. Natl. Acad. Sci. USA* **2025**, *122*, e2411962122. [CrossRef] [PubMed]
52. Sahay, R.; Eapen, B.; Meng, W.; Mamun, M.R.A.; Dora, N.K.; Sumasadan, M.; Tetarave, S.K.; Soto, R. Policy-Guided Threat Hunting: An LLM enabled Framework with Splunk SOC Triage. *arXiv* **2026**, arXiv:2603.23966. [CrossRef]
53. Tellache, A.; Korba, A.A.; Mokhtari, A.; Moldovan, H.; Ghamri-Doudane, Y. Advancing Autonomous Incident Response: Leveraging LLMs and Cyber Threat Intelligence. *arXiv* **2025**, arXiv:2508.10677. [CrossRef]
54. Huang, L.; Xiao, X. CTIKG: LLM-Powered Knowledge Graph Construction from Cyber Threat Intelligence. In Proceedings of the First Conference on Language Modeling (COLM 2024), Philadelphia, PA, USA, 7–9 October 2024.
55. Cheng, Y.; Bajaber, O.; Tsegai, S.A.; Song, D.; Gao, P. Ctinexus: Automatic cyber threat intelligence knowledge graph construction using large language models. In *Proceedings of the 2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*; IEEE: New York, NY, USA, 2025; pp. 923–938.
56. Jin, J.; Tang, B.; Ma, M.; Liu, X.; Wang, Y.; Lai, Q.; Yang, J.; Zhou, C. Crimson: Empowering strategic reasoning in cybersecurity through large language models. In *Proceedings of the 2024 5th International Conference on Computer, Big Data and Artificial Intelligence (ICCBD+ AI)*; IEEE: New York, NY, USA, 2024; pp. 18–24.
57. Paul, S.; Alemi, F.; Macwan, R. Llm-assisted proactive threat intelligence for automated reasoning. *arXiv* **2025**, arXiv:2504.00428. [CrossRef]
58. Tseng, P.; Yeh, Z.; Dai, X.; Liu, P. Using llms to automate threat intelligence analysis workflows in security operation centers. *arXiv* **2024**, arXiv:2407.13093. [CrossRef]
59. Xuan, Z.; Xu, X.; Zheng, M.; Tan, L.Z.H.; Guo, J.; Zhang, T.; Yu, L.; Wang, C.; Zhang, X. Identifying Adversary Tactics and Techniques in Malware Binaries with an LLM Agent. *arXiv* **2026**, arXiv:2602.06325. [CrossRef]
60. Alam, M.T.; Bhusal, D.; Ahmad, S.; Rastogi, N.; Worth, P. AthenaBench: A Dynamic Benchmark for Evaluating LLMs in Cyber Threat Intelligence. *arXiv* **2025**, arXiv:2511.01144. [CrossRef]
61. Cotti, L.; Drago, I.; Rula, A.; Bianchini, D.; Cerutti, F. OntoLogX: Ontology-guided knowledge graph extraction from cybersecurity logs with large language models. *Adv. Intell. Syst.* **2026**, *8*, e202501381. [CrossRef]
62. Meng, R.; Mirchev, M.; Böhme, M.; Roychoudhury, A. Large Language Model guided Protocol Fuzzing. In Proceedings of the Network and Distributed System Security (NDSS) Symposium 2024, San Diego, CA, USA, 26 February–1 March 2024.
63. Wang, C.; Zhang, W.; Su, Z.; Xu, X.; Xie, X.; Zhang, X. LLMDFA: Analyzing dataflow in code with large language models. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 131545–131574. [CrossRef]
64. Li, X.; Su, Y.; Liu, J.; Lin, Z.; Hou, Y.; Gao, P.; Zhang, Y. Vulsolver: Vulnerability detection via llm-driven constraint solving. *arXiv* **2025**, arXiv:2509.00882. [CrossRef]
65. Maklad, Y.; Wael, F.; Hamdi, A.; Elsersy, W.; Shaban, K. MultiFuzz: A dense retrieval-based multi-agent system for network protocol fuzzing. In *Proceedings of the 2025 IEEE/ACS 22nd International Conference on Computer Systems and Applications (AICCSA)*; IEEE: New York, NY, USA, 2025; pp. 1–8.
66. Mandiant. *M-Trends 2025: Data, Insights, and Recommendations from the Frontlines*; Technical Report; Google Cloud: Mountain View, CA, USA, 2025.
67. Yan, H.; Vaidya, S.S.; Zhang, X.; Yao, Z. Guiding ai to fix its own flaws: An empirical study on llm-driven secure code generation. *arXiv* **2025**, arXiv:2506.23034. [CrossRef]

68. Happe, A.; Cito, J. Getting pwn'd by ai: Penetration testing with large language models. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, CA, USA, 3–9 December 2023; pp. 2082–2086.
69. Happe, A.; Kaplan, A.; Cito, J. Llms as hackers: Autonomous linux privilege escalation attacks. *Empir. Softw. Eng.* **2026**, *31*, 70. [CrossRef]
70. Fang, R.; Bindu, R.; Gupta, A.; Zhan, Q.; Kang, D. Llm agents can autonomously hack websites. *arXiv* **2024**, arXiv:2402.06664. [CrossRef]
71. Fang, R.; Bindu, R.; Gupta, A.; Kang, D. LLM Agents Can Autonomously Exploit One-Day Vulnerabilities. *arXiv* **2024**, arXiv:2404.08144. [CrossRef]
72. Zhu, Y.; Kellermann, A.; Gupta, A.; Li, P.; Fang, R.; Bindu, R.; Kang, D. Teams of llm agents can exploit zero-day vulnerabilities. In Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), Rabat, Morocco, 24–29 March 2026; pp. 23–35.
73. Huang, H.; Shi, J.; Chen, J.; Zhang, T.; Li, Y.; Yang, C.; Ouh, E.L.; Shar, L.K.; Lo, D. PenForge: On-the-fly expert agent construction for automated penetration testing. In Proceedings of the IEEE/ACM 48th International Conference on Software Engineering, Rio de Janeiro, Brazil, 12–18 April 2026; pp. 76–80.
74. Wang, L.; Shi, X.; Li, Z.; Jiang, Y.; Tan, S.; Jiang, Y.; Cheng, J.; Chen, W.; Shen, X.; LI, Z.; et al. Automated Penetration Testing with LLM Agents and Classical Planning. *arXiv* **2025**, arXiv:2512.11143. [CrossRef]
75. Yang, R.; Cheng, M.; Deng, G.; Zhang, T.; Wang, J.; Xie, X. PentestEval: Benchmarking LLM-Based Penetration Testing with Modular and Stage-Level Design. *arXiv* **2025**, arXiv:2512.14233. [CrossRef]
76. Muzsai, L.; Imolai, D.; Lukács, A. Hacksynth: Llm agent and evaluation framework for autonomous penetration testing. *arXiv* **2024**, arXiv:2412.01778. [CrossRef]
77. Challita, B.; Parrend, P. RedTeamLLM: An Agentic AI Framework for Offensive Security. *arXiv* **2025**, arXiv:2505.06913. [CrossRef]
78. Luong, P.D.; Le Bao, T.G.; Khai Tam, N.V.; Nguyen Khoa, D.H.; Quyen, N.H.; Pham, V.H.; Duy, P.T. xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi agent systems. *arXiv* **2025**, arXiv:2509.13021. [CrossRef]
79. Kong, H.; Hu, D.; Ge, J.; Li, L.; Li, H.; Li, T. Pentest-r1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning. *arXiv* **2025**, arXiv:2508.07382. [CrossRef]
80. Zhuo, T.Y.; Wang, D.; Ding, H.; Kumar, V.; Wang, Z. Cyber-zero: Training cybersecurity agents without runtime. In Proceedings of the International Conference on Learning Representations, Rio de Janeiro, Brazil, 23–27 April 2026; Volume 2026, pp. 154054–154184.
81. Muzsai, L.; Imolai, D.; Lukács, A. Improving llm agents with reinforcement learning on cryptographic ctf challenges. *arXiv* **2025**, arXiv:2506.02048. [CrossRef]
82. Huang, Z.; Zhuge, J.; Zhuge, J. Multi-agent framework utilizing large language models for solving capture-the-flag challenges in cybersecurity competitions. *Appl. Sci.* **2025**, *15*, 7159. [CrossRef]
83. Shao, M.; Jancheska, S.; Udeshi, M.; Dolan-Gavitt, B.; Xi, H.; Milner, K.; Chen, B.; Yin, M.; Garg, S.; Krishnamurthy, P.; et al. Nyu ctf bench: A scalable open-source benchmark dataset for evaluating llms in offensive security. *Adv. Neural Inf. Process. Syst.* **2024**, *37*, 57472–57498. [CrossRef]
84. Rani, N.; Milner, K.; Shao, M.; Udeshi, M.; Xi, H.; Putrevu, V.S.C.; Aggarwal, S.; Shukla, S.K.; Krishnamurthy, P.; Khorrami, F.; et al. CTFExplorer: Evaluating LLM Offensive Agents Through Multi-Target Web CTF Benchmarking. *arXiv* **2026**, arXiv:2602.08023. [CrossRef]
85. Huang, L.; Dave, D.; Cody, T.; Beling, P.A.; Jin, M. From capabilities to performance: Evaluating key functional properties of llm architectures in penetration testing. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Suzhou, China, 4–9 November 2025; pp. 15890–15916.
86. Somma, M.; Großpointner, M.; Zabalegui, P.; Heilimo, E.; Stojanović, B. Environment-Grounded Multi-Agent Workflow for Autonomous Penetration Testing. In *Proceedings of the 2026 IEEE 12th International Conference on Network Softwarization (NetSoft)*; IEEE: New York, NY, USA, 2026; pp. 570–576.
87. Goyal, D.; Subramanian, S.; Peela, A.; Shetty, N.P. Hacking, the lazy way: LLM augmented pentesting. *arXiv* **2024**, arXiv:2409.09493. [CrossRef]
88. Saha, B.; Shukla, S.K. MalGEN: A Generative Agent Framework for Modeling Malicious Software in Cybersecurity. *arXiv* **2025**, arXiv:2506.07586. [CrossRef]
89. Ayzenshteyn, D.; Weiss, R.; Mirsky, Y. The best defense is a good offense: Countering llm-powered cyberattacks. *arXiv* **2024**, arXiv:2410.15396. [CrossRef]
90. Liu, Z. Multi-Agent Collaboration in Incident Response with Large Language Models. *arXiv* **2024**, arXiv:2412.00652. [CrossRef]
91. Xiao, Z.; Sun, J.; Chen, J. AIR: Improving Agent Safety Through Incident Response. In Proceedings of the 43rd International Conference on Machine Learning (ICML), Seoul, Republic of Korea, 6–11 July 2026; Proceedings of Machine Learning Research; Volume 306.

92. European Banking Authority. *Rising Application of AI in EU Banking and Payments Sector*; Technical Report; European Banking Authority: Paris, France, 2025. Available online: https://www.eba.europa.eu/sites/default/files/2025-09/146b3558-d026-47bf-a872-f05e93ed30d2/Rising%20application%20of%20AI%20in%20EU%20banking%20and%20payments%20sector.pdf (accessed on 19 August 2026).
93. Li, Y.; Huang, C.; Deng, S.; Lock, M.L.; Cao, T.; Oo, N.; Lim, H.W.; Hooi, B. KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; pp. 793–810.
94. Xue, Y.; Spero, E.; Koh, Y.S.; Russello, G. MultiPhishGuard: An LLM-based multi-agent system for phishing email detection. *arXiv* **2025**, arXiv:2505.23803. [CrossRef]
95. Ginige, Y.; Niroshan, A.; Jain, S.; Seneviratne, S. AutoPentester: An LLM Agent-based Framework for Automated Pentesting. *arXiv* **2025**, arXiv:2510.05605. [CrossRef]
96. Xu, J.; Stokes, J.W.; McDonald, G.; Bai, X.; Marshall, D.; Wang, S.; Swaminathan, A.; Li, Z. Autoattacker: A large language model guided system to implement automatic cyber-attacks. *arXiv* **2024**, arXiv:2403.01038. [CrossRef]
97. Nguyen, V.K.; Husain, M.I. Penetration Testing of Agentic AI: A Comparative Security Analysis Across Models and Frameworks. *arXiv* **2025**, arXiv:2512.14860. [CrossRef]
98. Gioacchini, L.; Mellia, M.; Drago, I.; Delsanto, A.; Siracusano, G.; Bifulco, R. AutoPenBench: Benchmarking Generative Agents for Penetration Testing. *arXiv* **2024**, arXiv:2410.03225. [CrossRef]
99. Bhatt, M.; Chennabasappa, S.; Nikolaidis, C.; Wan, S.; Evtimov, I.; Gabi, D.; Song, D.; Ahmad, F.; Aschermann, C.; Fontana, L.; et al. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. *arXiv* **2023**, arXiv:2312.04724. [CrossRef]
100. *ISO 20022-1:2013*; Financial Services—Universal Financial Industry Message Scheme—Part 1: Metamodel. International Organization for Standardization: Geneva, Switzerland, 2013.
101. Wu, B.; Chen, G.; Chen, K.; Shang, X.; Han, J.; He, Y.; Zhang, W.; Yu, N. Autopt: How far are we from the end2end automated web penetration testing? *arXiv* **2024**, arXiv:2411.01236. [CrossRef]
102. Abramovich, T.; Udeshi, M.; Shao, M.; Lieret, K.; Xi, H.; Milner, K.; Jancheska, S.; Yang, J.; Jimenez, C.E.; Khorrami, F.; et al. Enigma: Enhanced interactive generative model agent for ctf challenges. *arXiv* **2024**, arXiv:2409.16165. [CrossRef]
103. Shen, X.; Wang, L.; Li, Z.; Chen, Y.; Zhao, W.; Sun, D.; Wang, J.; Ruan, W. PentestAgent: Incorporating LLM Agents to Automated Penetration Testing. In *Proceedings of the ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*; ACM: New York, NY, USA, 2025. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.